

**LICENCIATURA EM  
CIÊNCIAS DA COMPUTAÇÃO**

# **PROGRAMAÇÃO II**

PRESIDENTE DA REPÚBLICA

**Dilma Roussef**

MINISTRO DA EDUCAÇÃO

**Aloísio Mercadante**

SISTEMA UNIVERSIDADE ABERTA DO BRASIL

PRESIDENTE DA CAPES

**Jorge Guimarães**

DIRETOR DE EDUCAÇÃO A DISTÂNCIA DA CAPES

**João Teatini**

GOVERNO DO ESTADO DA BAHIA

GOVERNADOR

**Jaques Wagner**

VICE-GOVERNADOR

**Otto Roberto Mendonça de Alencar**

SECRETÁRIO DA EDUCAÇÃO

**Osvaldo Barreto Filho**

UNIVERSIDADE DO ESTADO DA BAHIA - UNEB

REITOR

**Lourivaldo Valentim da Silva**

VICE-REITORA

**Adriana do Santos Marmori Lima**

PRÓ-REITOR DE ENSINO DE GRADUAÇÃO

**José Bites de Carvalho**

COORDENADOR UAB/UNEB

**Silvar Ferreira Ribeiro**

COORDENADOR UAB/UNEB ADJUNTO

**André Magalhães**

**EDUARDO MANUEL DE FREITAS JORGE**  
**EDUARDO TELMO FONSECA SANTOS**  
**UEDSON SANTOS REIS**

**LICENCIATURA EM**  
**CIÊNCIAS DA COMPUTAÇÃO**

# **PROGRAMAÇÃO II**

**EDUNEB**  
**Salvador**

© UNEB 2012

Nenhuma parte deste material poderá ser reproduzida, transmitida ou gravada, por qualquer meio eletrônico, mecânico, por fotocópia e outros, sem a prévia autorização, por escrito, da Coordenação UAB/UNEB.

Depósito Legal na Biblioteca Nacional

Impresso no Brasil 2012

## **EDITORA DA UNIVERSIDADE DO ESTADO DA BAHIA**

### **DIRETORA**

Maria Nadjá Nunes Bittencourt

### **COORDENADOR EDITORIAL**

Ricardo Baroud

### **COORDENAÇÃO UAB/UNEB**

#### **COLABORADORES**

#### **SUPERVISÃO DE MATERIAL DIDÁTICO**

Andréa Santos Tanure

Flávia Souza dos Santos

Tatiane Nogueira Nunes

#### **PROJETO GRÁFICO e CRIAÇÃO**

João Victor Souza Dourado

Carla Cristiani Honorato de Souza

#### **REVISORES**

Carla Honorato

Maíra Andrade

#### **NORMALIZAÇÃO**

Sheila Rangel

#### **DIAGRAMAÇÃO**

Alan Venicius de Araújo Gonçalves

O conteúdo deste Material Didático é de inteira responsabilidade do(s)/da(s) autores (as), por cuja criação assume(m) ampla e total responsabilidade quanto a titularidade, originalidade do conteúdo intelectual produzido, uso de citações de obras consultadas, referências, imagens e outros elementos que façam parte desta publicação.

---

J82 JORGE, Eduardo Manuel de Freitas; SANTOS, Eduardo Telmo Fonseca; REIS, Uedson Santos.  
Programação II: bacharelado em ciência da computação. / Eduardo Manuel de Freitas Jorge;  
Eduardo Telmo Fonseca Santos; Uedson Santos Reis. Salvador: UNEB/GEAD, 2013.  
72 p.

1. Programação 2. Ciência da Computação I. Eduardo Manuel de Freitas Jorge II. Eduardo  
Telmo Fonseca Santos III. Título. III. Uedson Santos Reis IV. Universidade Aberta do Brasil. V.  
UNEB / GEAD

CDD: 658.05

---



Editora da Universidade do Estado da Bahia - EDUNEB.

Rua Silveira Martins, 2555 - Cabula

41150-000 - Salvador - BA

www.eduneb.uneb.br

editora@listas.uneb.br Tel. + 55 71 3117-5342

**LICENCIATURA EM CIÊNCIAS DA COMPUTAÇÃO**

## Caro (a) Cursista

Estamos começando uma nova etapa de trabalho e para auxiliá-lo no desenvolvimento da sua aprendizagem estruturamos este material didático que atenderá ao Curso de Licenciatura na modalidade de Educação a Distância (EaD).

O componente curricular que agora lhe apresentamos foi preparado por profissionais habilitados, especialistas da área, pesquisadores, docentes que tiveram a preocupação em alinhar o conhecimento teórico e prático de maneira contextualizada, fazendo uso de uma linguagem motivacional, capaz de aprofundar o conhecimento prévio dos envolvidos com a disciplina em questão. Cabe salientar, porém, que esse não deve ser o único material a ser utilizado na disciplina, além dele, o Ambiente Virtual de Aprendizagem (AVA), as atividades propostas pelo Professor Formador e pelo Tutor, as atividades complementares, os horários destinados aos estudos individuais, tudo isso somado compõe os estudos relacionados à EaD.

É importante também que vocês estejam sempre atentos às caixas de diálogos e ícones específicos que aparecem durante todo o texto apresentando informações complementares ao conteúdo. A ideia é mediar junto ao leitor, uma forma de dialogar questões para o aprofundamento dos assuntos, a fim de que o mesmo se torne interlocutor ativo desse material.

São objetivos dos ícones em destaque:



### VOCÊ SABIA?

– convida o leitor a conhecer outros aspectos daquele tema/ conteúdo. São curiosidades ou informações relevantes que podem ser associadas à discussão proposta.



### SAIBA MAIS

– apresenta notas, textos para aprofundamento de assuntos diversos e desenvolvimento da argumentação, conceitos, fatos, biografias, enfim, elementos que o auxiliam a compreender melhor o conteúdo abordado.



### INDICAÇÃO DE LEITURA

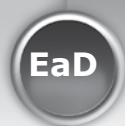
– neste campo, você encontrará sugestões de livros, sites, vídeos. A partir deles, você poderá aprofundar seu estudo, conhecer melhor determinadas perspectivas teóricas ou outros olhares e interpretações sobre determinado tema.



### SUGESTÃO DE ATIVIDADE

– consiste num conjunto de atividades para você realizar autonomamente em seu processo de autoestudo. Estas atividades podem (ou não) ser aproveitadas pelo professor-formador como instrumentos de avaliação, mas o objetivo principal é o de provocá-lo, desafiá-lo em seu processo de autoaprendizagem.

Sua postura será essencial para o aproveitamento completo desta disciplina. Contamos com seu empenho e entusiasmo para juntos desenvolvermos uma prática pedagógica significativa.



## ÁREA DE AMBIENTAÇÃO

### Dados Pessoais

Nome:

Telefones:

E-mail:

Residência:

Município/Polo do curso EaD:

### Dados do Curso

·Endereço de acesso à sala de aula virtual:

·Período de execução desta disciplina: de \_\_\_\_\_ à \_\_\_\_/\_\_\_\_/\_\_\_\_

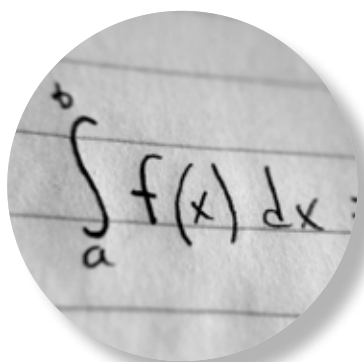
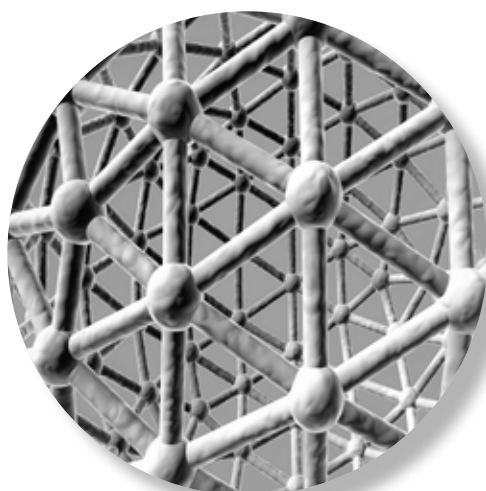
·Quem são os orientadores desta disciplina?

| Categoria         | Responsável                         | por Nome | E-mail |
|-------------------|-------------------------------------|----------|--------|
| Prof. Autor       | Elaborar o módulo impresso          |          |        |
| Prof. Formador    | Planejar e organizar a sala virtual |          |        |
| Tutor a distância | Mediar os estudos no ambiente       |          |        |
| Tutor presencial  | Mediar os encontros presenciais     |          |        |
| Coord. de Polo    | Apoiar as ações do curso no local   |          |        |

### Identificação da Turma

| Nome | Onde encontrar | Telefones | E-mail |
|------|----------------|-----------|--------|
|      |                |           |        |
|      |                |           |        |

## APRESENTAÇÃO



Prezado (a) estudante,

Vamos iniciar os estudos da disciplina de Programação II, cujo objetivo é desenvolver sua capacidade de resolução de problemas através do paradigma da OO (Orientação a Objetos) .

Nesta disciplina de 60 horas, veremos um breve histórico que contextualiza o surgimento e evolução do paradigma OO. Além disso, os fundamentos básicos, tais como: Classes, Objetos, Métodos, Atributos, Encapsulamento, Herança de Classe, Generalização/Especialização, Classe Abstrata/Método Abstrato, Método Construtor e Polimorfismo serão trabalhados com vocês, objetivando a compreensão de como é possível fazer representações de mundo em OO. Em um segundo momento, veremos como colocar em prática o paradigma OO através de uma linguagem de programação. Lembrando de que a grande fórmula de estudo, neste caso, é a resolução de muitos exercícios. Usaremos a linguagem de programação Java para isso, mas sem o objetivo de trabalhar completamente todos os recursos da mesma. Usaremos a estratégia de níveis para, a cada exemplo e exercício, ampliarmos, paulatinamente, a complexidade. Assim, você chegará ao final da disciplina capacitado e confiante neste paradigma de programação.

Codificar um problema do nosso dia a dia em uma linguagem que o computador entenda é desafiador e empolgante. O segredo é simplificar e reduzir o problema, focando a resolução em etapas e lidando, primeiramente, com suas questões essenciais. Além disso, deve-se ter conhecimento da linguagem que o computador entende. Logo, visa-se transformar o problema em um algoritmo que pode ser executado em uma máquina. Um dos desafios iniciais é criar uma excelente ambientação com a linguagem que o computador entende. Para tanto, a resolução de vários exercícios irá lhe proporcionar isso,

chegando ao ponto em que você vai pensar na solução quase que automaticamente. Além do conteúdo aqui apresentado, esperamos que este material sirva para ajudá-los, sempre que necessário, no ensino da disciplina de programação.

O desenvolvimento de projetos de softwares não é uma tarefa fácil. Os projetos, geralmente, têm que ser robustos, atuar sobre problemas complexos e estar implantados em curto prazo (time-to-market). Entretanto, é necessário produzir softwares genéricos e extensíveis que possam ser aplicados a uma gama de aplicações. Ao longo dos anos, muitas metodologias surgiram na Engenharia de Software com o intuito de reduzir a complexidade e aumentar a produtividade no desenvolvimento de software. A Orientação a Objetos é reconhecida, hoje, como o principal paradigma para atender a esses requisitos. Assim, conhecimentos em fundamentos nesta área estão na essência da formação de um profissional que deseja atuar na área de Informática.

Este livro aborda os conceitos da Orientação a Objetos através da interação entre a teoria e a prática. Está no escopo do conteúdo do livro uma introdução à Engenharia de Software abordando técnicas como Programação Orientada a Objetos, Padrões de Projeto, Framework e Testes de Software.

Observa-se que desenvolvedores e interessados na produção de software devem estruturar os seus conhecimentos, mesmo que sejam iniciantes na área da Ciência da Computação, em princípios e boas práticas da Engenharia de Software. Questões como reutilização, arquitetura em camadas, delegação clara de responsabilidades, modularização, redução de acoplamento e complexidade, teste de software são alguns dos princípios que irão nortear toda a apresentação do conteúdo deste livro.

Este é um livro que, apesar de demonstrar os seus exemplos de programação na linguagem Java, não tem como objetivo a apresentação desta linguagem em detalhes. O intuito disto é fugir dos livros que pecam nos fundamentos teóricos e estão mais preocupados com a apresentação de técnicas de linguagens como Java, PHP, C++ entre outras. Assim, aborda-se a linguagem Java somente para apresentar o conteúdo relevante para o entendimento dos exemplos que facilitam o entendimento dos fundamentos da Orientação a Objetos. Conhecimentos mais específicos da linguagem Java sobre tipos de dados e comandos devem ser obtidos através de literaturas complementares.

O método de apresentação do conteúdo do livro consiste, primeiramente, em tratar dos fundamentos teóricos básicos e, posteriormente, serão apresentados exemplos práticos para facilitar e reforçar os conceitos inicialmente vistos. Em seguida, será apresentado o conteúdo mesclando a teoria e a prática. Para facilitar a tarefa de programação Orientada a Objetos, os exemplos de algoritmos estão em uma ordem crescente de complexidade, ou seja, cada exemplo acrescenta novos aspectos ligados a este paradigma de desenvolvimento. A motivação para o estudo do conteúdo deste livro é entender a necessidade para a uma boa formação dos conceitos aqui apresentados, pois os mesmos são a base para a obtenção de diversos outros conhecimentos imprescindíveis para a realização de projetos em desenvolvimento de software.

Existe uma separação clara dos profissionais que possuem esta base e, assim, estão aptos para assimilar e construir, até de forma autodidata, softwares bem estruturados. No contexto da internet e dos dispositivos

móveis, é imprescindível ter conhecimento sobre tecnologias de desenvolvimento Orientado a Objetos para a construção de Sistemas Web, Sistemas Embarcados, Sistemas Distribuídos, Programação para Dispositivos Móveis etc.

Neste cenário, o profissional reutiliza componentes e técnicas que reduzem o ciclo de vida de construção de software e aumentam a qualidade dos projetos, questões fundamentais para o sucesso do mesmo.

A transposição da fase inicial de entendimento dos conceitos sobre Orientação a Objetos pode parecer difícil no seu começo devido a mudanças de paradigma e a necessidade de bons fundamentos para resolverem problemas e codificá-los em uma linguagem. Para os que já programam no paradigma funcional, existe a necessidade de reestruturar o pensamento da forma de como é feita a construção de software, ou seja, é necessária uma quebra de paradigma em relação à forma de como se projeta software convencional.

Percebe-se que, com a assimilação dos conceitos de Orientação a Objetos, dificilmente volta-se a pensar na resolução de problemas usando o paradigma funcional. Ressalta-se que alguns algoritmos com poucas linhas de código podem ficar mais difíceis de serem resolvidos na programação Orientada a Objetos. Porém, como os softwares nas corporações são normalmente de médio ou grande porte, a Orientação a Objetos é fundamental para futuras manutenções corretivas e evolutivas. Bons estudos e aproveitem bastante o conteúdo deste material.



#### INDICAÇÃO DE LEITURA

Este material usa como base nos seus exemplos práticos, em todas as suas seções, o livro “Core Java 2: Fundamentos (vol.1.)”. Este é um clássico de referência em Programação OO dos autores Gary Cornell. Assim, recomenda-se que você complemente os seus conhecimentos com a leitura deste livro.

Bons Estudos.

Eduardo M F Jorge, Eduardo Telmo e Uedson Reis



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

# SUMÁRIO

|                                                                                                                    |           |
|--------------------------------------------------------------------------------------------------------------------|-----------|
| <b>CAPÍTULO 1 – QUESTÕES HISTÓRICAS DA ORIENTAÇÃO A OBJETOS .....</b>                                              | <b>13</b> |
| <b>CAPÍTULO 2 – CONCEITOS BÁSICOS OO.....</b>                                                                      | <b>17</b> |
| 2.1 Objeto .....                                                                                                   | 19        |
| 2.2 Classe.....                                                                                                    | 19        |
| 2.3 Herança de classe .....                                                                                        | 22        |
| 2.4 Métodos, método construtor e sobrecarga.....                                                                   | 23        |
| 2.5 Polimorfismo.....                                                                                              | 23        |
| 2.6 Coesão .....                                                                                                   | 24        |
| <b>CAPÍTULO 3 – APLICANDO OS CONCEITOS DE OBJETO, CLASSE, ENCAPSULAMENTO E SOBRECARGA NA LINGUAGEM JAVA.....</b>   | <b>27</b> |
| 3.1 Exemplo 1: primeiro programa oo .....                                                                          | 30        |
| 3.2 Exemplo 2: evoluindo o exemplo 1 aplicando os conceitos de encapsulamento .....                                | 31        |
| 3.3 Exemplo 3: exemplificando método construtor e sobrecarga .....                                                 | 31        |
| 3.4 Herança, polimorfismo e cast.....                                                                              | 32        |
| 3.4.1 Exemplo 4: exemplificando herança .....                                                                      | 32        |
| 3.4.2 Exemplo 5: exemplificando herança - 2 .....                                                                  | 33        |
| 3.4.3 Exemplo 6: exemplificando classe abstrata e polimorfismo.....                                                | 34        |
| <b>CAPÍTULO 4 – VANTAGENS E DIFERENÇAS ENTRE A PROGRAMAÇÃO FUNCIONAL E A PROGRAMAÇÃO ORIENTADA A OBJETOS .....</b> | <b>37</b> |
| 4.1 Benefícios alcançados com a oo: acoplamento e complexidade, quanto menor melhor.....                           | 40        |
| 4.2 Exemplo prático em java de comparativo de programação oo versus programação f. Estruturada .....               | 42        |
| <b>CAPÍTULO 5 – ASSOCIAÇÕES E COMPOSIÇÃO .....</b>                                                                 | <b>47</b> |
| 5.1 Componente.....                                                                                                | 49        |
| 5.2 Aplicando conceitos relacionados com composição .....                                                          | 49        |
| 5.3 Coleções (collection).....                                                                                     | 50        |
| 5.4 Motivação para a reutilização de software.....                                                                 | 53        |
| 5.5 Reutilização e orientação a objetos .....                                                                      | 54        |
| 5.6 Framework orientado a objetos .....                                                                            | 55        |
| 5.7 Conceito de framework.....                                                                                     | 55        |
| 5.8 Framework versus biblioteca de classes .....                                                                   | 55        |
| 5.9 Framework e reutilização de software .....                                                                     | 56        |
| 5.10 Construindo aplicações a partir de um framework.....                                                          | 57        |
| 5.11 Framework “a bola da vez” .....                                                                               | 57        |
| <b>CAPÍTULO 6 – PADRÕES DE PROJETO .....</b>                                                                       | <b>59</b> |
| 6.1 Padrões de projeto e framework .....                                                                           | 61        |
| 6.2 Template method.....                                                                                           | 62        |
| <b>CAPÍTULO 7 – O PROCESSO UNIFICADO .....</b>                                                                     | <b>65</b> |
| 7.1 Desenvolvimento orientado por caso de uso .....                                                                | 67        |
| 7.2 Ênfase na arquitetura ( desde a fase de elaboração do projeto).....                                            | 68        |
| 7.3 Abordagem iterativa e incremental .....                                                                        | 68        |
| <b>REFERÊNCIAS .....</b>                                                                                           | <b>71</b> |



## Anotações

A large, empty rectangular area with rounded corners, intended for taking notes. It has a light gray border and a subtle drop shadow, giving it a three-dimensional appearance as if it's a sheet of paper floating on the page.

**QUESTÕES  
HISTÓRICAS DA  
ORIENTAÇÃO A  
OBJETOS**

# **CAPÍTULO 1**



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

Métodos de modelagem Orientados a Objetos começaram a aparecer entre meados da década 70 e início dos anos 80. Os primeiros registros da concepção dos conceitos de Orientação a Objetos surgiram com a linguagem Simula na década de 60, na Noruega, no centro de Norweigan Computin Center (NCC), sobre a responsabilidade de Kristen Nygaard e Ole-Johan Dahl. O cerne dessa linguagem era a criação de um modelo computacional que fosse similar às estruturas do mundo real, facilitando, assim, o projeto de sistemas. Após a linguagem Simula, um outro projeto a implementar os conceitos de Orientação a Objetos muito conhecido foi a linguagem Smalltalk, criada pelo núcleo de pesquisa da Xerox PARC e desenvolvida pelos pesquisadores Adele Goldberg, Alan Key e Dan Ingalls. Estes métodos e linguagens, inicialmente, permearam o meio acadêmico em pesquisas pouco aplicáveis.

Outra linguagem importante na história da Orientação a Objetos foi a linguagem C++. Esta é uma evolução da linguagem C que está sobre o paradigma Estruturado Funcional. A primeira versão foi desenvolvida em 1980 com o nome de 'C with Classes'. Em 1983 é lançada a primeira versão do C++ com diversas características de Orientação a Objetos. O pesquisador responsável por essa evolução da linguagem C foi Bjarne Stoustrup. Apesar da linguagem C++ ser poderosa, alguns pesquisadores e professores têm restrições quanto ao primeiro contato do aluno, com os conceitos de Orientação a Objetos, ser com a linguagem C++. Um dos principais motivos dessa restrição deve-se ao fato do C++ ser uma linguagem híbrida. Como foi dito, o C++ é uma evolução da linguagem C que está sobre o paradigma Estruturado Funcional, permitindo que o aluno programe desrespeitando os conceitos da Orientação a Objetos.

Já a linguagem Java surgiu no ano de 1991, projetada por James Gosling, Bill Joy e Guy Steele na Sun Microsystems. O seu projeto inicial visava ser uma linguagem para programação de sistemas embarcados. Estes programas rodariam em dispositivos eletrônicos da Sun Microsystem. Com o surgimento da Internet e da WWW (*World Wide Web*), a linguagem Java começou a

ser amplamente utilizada. O sucesso da linguagem está associado com o recurso de independência de plataforma e por ser uma linguagem parecida e mais simples que C e C++, visto que a alocação e a liberação de memória não são feitos diretamente pelo programador. Portanto, Java possui um mecanismo de gerência de memória e coleta de lixo que evitam erros de manipulação de memória. Existem outras linguagens de programação, porém menos conhecidas.

Na década de 90, a aplicação comercial dos conceitos da Orientação a Objetos se fortaleceu, saindo do meio acadêmico e sendo aplicados nas organizações no desenvolvimento de suas soluções de software comerciais. Uma das causas dessa popularização foi o aparecimento das ferramentas RAD (*Rapid Application Development*), que objetivavam aumentar a produtividade na construção de software. Estas ferramentas eram aplicadas, principalmente, na construção de elementos da camada de interface, tais como menus, botões, telas, tabelas, combos, etc. Os mecanismos de construção de aplicações para o desenvolvedor na ferramenta RAD se dá através da reutilização de componentes visuais, previamente construídos, que são manipulados e customizados para atender aos requisitos específicos da aplicação que está sendo construída. Um exemplo clássico de ferramentas RAD são as IDE Visual Basic, Delphi, Centura etc. A figura1 ilustra a IDE Visual Básica com a barra de componentes à esquerda, e à direita a lista de propriedades que podem ser configuradas para os componentes.

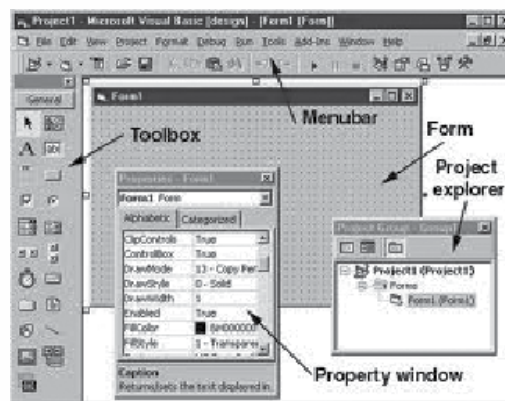


Figura1- IDE Visual Basic

Fonte: Os autores

Apesar dos benefícios alcançados com a produtividade e facilidade no desenvolvimento de software, diversos problemas relacionados com a construção de aplicações baseados em RAD são identificados. Uma primeira questão a ser analisada é que um software normalmente possui um custo muito mais alto de manutenção do que de desenvolvimento. Como as ferramentas RAD e as técnicas de Engenharia de Software difundidas na década de 90, os desenvolvedores acabavam construindo softwares com um forte acoplamento entre a camada de aplicação e a camada de interface (ver figura2). Este tipo de solução gerava graves problemas de acoplamento e baixa reutilização dificultando a manutenção corretiva e evolutiva das aplicações. Por exemplo, o espalhamento de regras de negócio nas telas exige uma mudança em vários pontos do software.

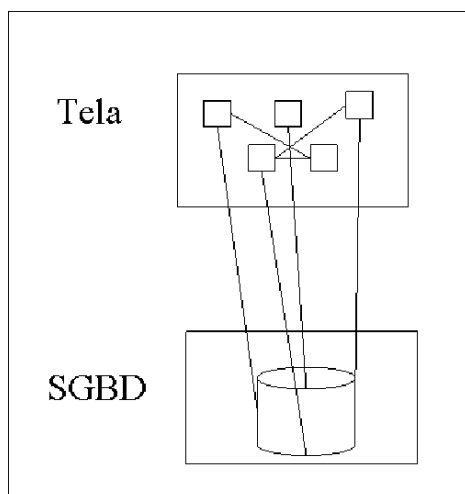


Figura 2 - Exemplo de desenvolvimento com forte acoplamento entre a camada de interface e camada de persistência  
Fonte: Os autores

Na Ciência da Computação, a modelagem Orientada a Objeto se popularizou na década de 90 com o sucesso da linguagem de notação UML (*The Unified Modeling Language*) que unificou as melhores práticas da OMT (*Object Modeling Technique*) proposta por Booch, Rumbaugh e modelo OOSE (*Object-Oriented Software Engineering*) proposta por Jacobson. A UML definida como uma linguagem unificada para modelagem gráfica de objetos foi um marco na Engenharia de Software, pois antes do seu surgimento cada autor de referência na área tinha a sua forma de modelagem específica com alguns poucos elementos em comum. A UML tem uma grande aceitação na Engenharia de Software e já extrapola o

seu uso em outras áreas. Com isso, é uma estratégia interessante para modelagem de conceitos.

A Orientação a Objetos está ainda em plena evolução, apesar de ser um paradigma de desenvolvimento da década 1970. Exemplo são as técnicas de desenvolvimento baseadas em: Framework, Padrões de Projeto, Orientação a Aspectos, TDD (*Test Driven Development*), BDD (*Behavior Driven Development*) entre outras que agregam novas possibilidades a Orientação a Objetos.



#### INDICAÇÃO DE LEITURA

Vamos entender melhor dessa história tão rica do surgimento do paradigma OO. Para aprofundar esse conteúdo acesse ARONOVICH, Alberto Chaia; PORTO, Sérgio Ribeiro. **Orientação a objetos**: uma visão para o negócio. Disponível em: <<http://www.carto.eng.uerj.br/fgeorj/segeo1996/129/index.htm>>. Acesso em: 02 jul. 2013.

**CONCEITOS  
BÁSICOS 00**

# **CAPÍTULO 2**



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

Esta seção apresenta os conceitos iniciais sobre Orientação a Objetos. Estes conhecimentos são de suma importância, pois representam a base para a compreensão dos demais assuntos apresentados ao longo deste livro.

## 2.1 OBJETO

No paradigma da Orientação a Objetos, a principal estrutura de representação é o Objeto. Semanticamente é uma forma de mapear coisas do mundo real com estruturas internas de um programa de computador. Logo, o conceito Objeto é definido como qualquer elemento ou 'coisa' do mundo real. Uma definição mais formal pode ser conceituada da seguinte forma: um objeto é uma abstração de estruturas do mundo real.

Por exemplo, uma pessoa, um relógio, um produto são exemplos de objetos. Todo objeto é composto pelo seu estado que é um conjunto de dados que o caracterizam.

Imaginemos um objeto, o carro, com o seguinte estado: uma placa XPT 36789, a informação da quantidade de 40 litros no tanque e o consumo de 10 km/litros.

Um objeto também possui ciclo de vida, ou seja, ele nasce, modifica o seu estado ao longo do tempo e morre. A modificação de um objeto no decorrer do seu ciclo de vida representa uma mudança no seu estado ou transição de estado. Similar a um elemento do mundo real, um objeto é único por definição. Portanto, um objeto Carro pode até ter o mesmo estado de outro objeto Carro, mas são objetos diferentes por definição. Cada um possui o seu ciclo de vida próprio, portanto nascem, mudam de estado e morrem de forma particular. O Objeto é uma unidade semântica que agrupa dados, além disso possui um conjunto de comportamentos.

Comportamentos são mecanismos que, normalmente baseados no estado, permitem que o objeto faça algo e/ou responda sobre uma determinada ação. A mudança de estado de um objeto é normalmente realizada através de um comportamento. Por exemplo, a redução da

quantidade de litros no tanque do Carro é uma mudança de estado efetuada por um comportamento do próprio objeto. Em Programação Orientada a Objetos, o comportamento é escrito através de funções que são denominadas de métodos do objeto.



### VOCÊ SABIA?

Por exemplo, um objeto Carro pode possuir o método `double getAutonomiaKm()` para responder sobre a sua autonomia. Esta ação é processada sobre a responsabilidade do objeto Carro que, através do seu estado, pode responder a esse questionamento. Assim, com base no exemplo anterior, para saber a autonomia, seria feito o cálculo da (Quantidade de litros x Consumo), então  $(40 \text{ litros} \times 10 \text{ km/litros}) = 400 \text{ km}$ .

Ainda sobre Objeto, ele é um módulo indivisível ou atômico com métodos (comportamento) e o seu estado são variáveis que mapeiam informações compartilhadas por todos os métodos. Os métodos e o estado de um objeto são agrupados, no mesmo local, sobre uma semântica única, ou seja, em um único objeto.

## 2.2 CLASSE

Um conjunto de objetos do mesmo tipo é denominado de Classe. Define-se como uma forma natural de agrupar os objetos que possuem as mesmas características. Diferente do objeto, a Classe não possui estado, assim, pode-se conceituar Classe como uma estrutura estática utilizada para a construção de objetos (ver Figura 3).

Quando um Objeto é gerado a partir de uma Classe este herda todas as características da Classe, adicionando o seu estado particular. Assim, uma Classe determina os padrões estruturais do objeto. Uma Classe é constituída de atributos (variáveis compartilhadas) e de comportamentos representados pelas funções denominadas na concepção de um Objeto como métodos.

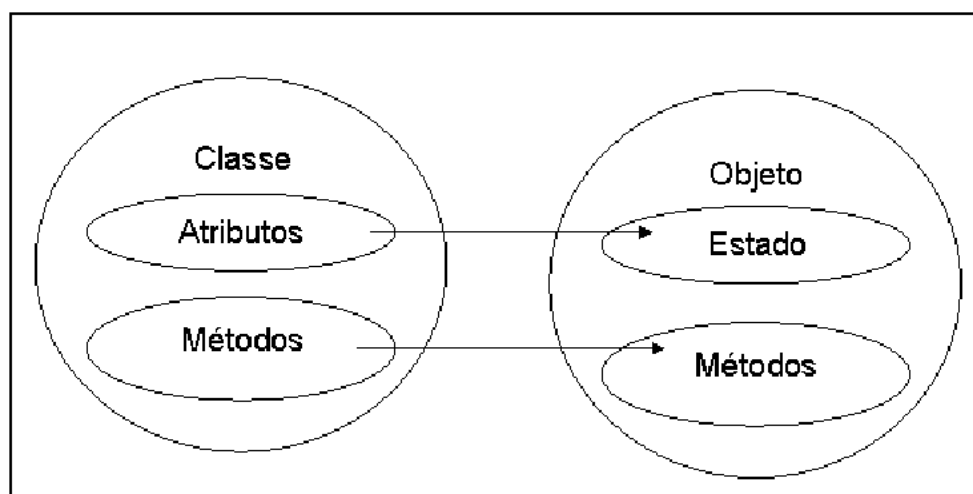


Figura 3-Representação de Classe e Objeto

Fonte: Os autores

No exemplo do objeto Carro, pode-se definir a classe Carro com os atributos placa, combustível, consumo e o comportamento representado através do método `double getAutonomiaKm()`. Sobre o questionamento de quem nasce primeiro um objeto ou uma classe, ressalta-se que é necessário, primeiro, a definição da Classe para, posteriormente, efetuar-se a geração dos objetos com base na Classe. Assim, entende-se também o conceito Classe como uma fábrica de objetos do mesmo tipo.

A criação de um objeto com base em uma Classe também é conhecida com a nomenclatura instância de Classe. Para ilustrar o conceito de classe, pode-se fazer

uma analogia com a planta de uma casa (ver Figura 4). Com base nesta planta, são construídas n casas (objetos) com as mesmas características básicas, diferenciando-as através do seu estado particular, por exemplo: cada casa pode ter uma cor diferenciada.



## SAIBA MAIS

Através da análise comparativa entre Classe e Objeto, concluem-se os seguintes pontos:

- Classe é uma estrutura estática composta por atributos e métodos.
- Uma Classe tem como funcionalidade principal ser uma estrutura genérica para a construção de objetos, ou seja, uma fábrica de objetos.
- Um objeto é gerado sempre a partir de uma Classe (instância de classe), herdando todas as suas características atributos e métodos, adicionando mais o estado.
- O estado do objeto é entendido como os valores assumidos ao longo do tempo pelos atributos, sendo utilizados como os dados para os métodos.

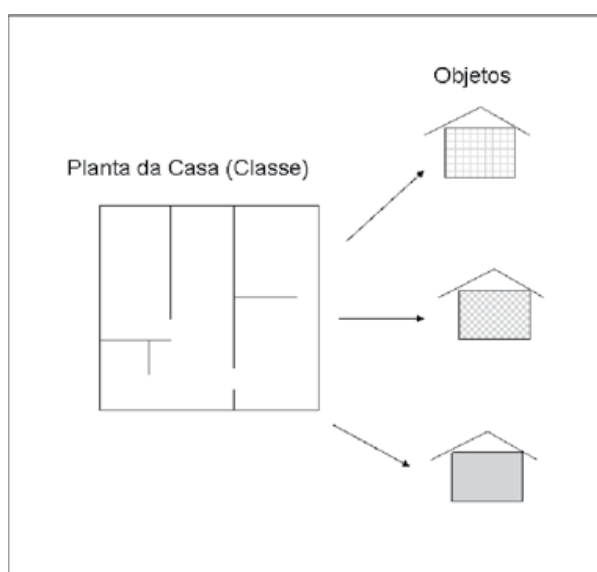


Figura 4 - Representação lúdica para o conceito de classe

Fonte: Os autores

Fortemente relacionado com o Conceito Classe e Objeto, temos o encapsulamento ou encapsulação que significa esconder informação. Objetiva-se com o encapsulamento proteger o estado e o comportamento interno do objeto, disponibilizando somente uma camada de interface com o ambiente externo para que seja efetuada a comunicação com o objeto, como ilustrado na Figura 5.

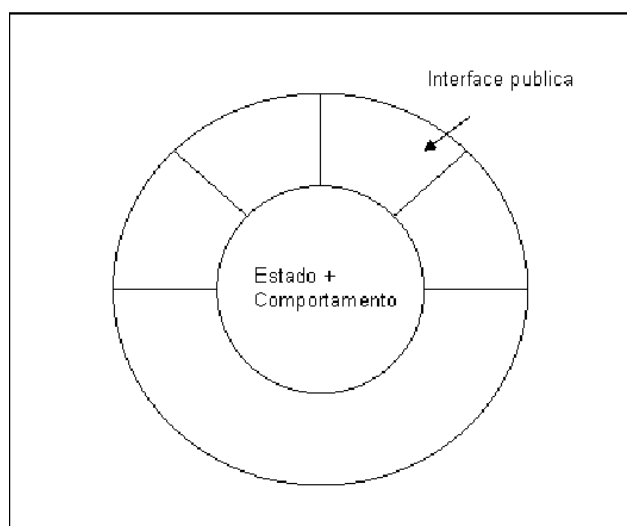


Figura 5 - Representação gráfica de um objeto  
Fonte: Os autores

O reflexo do encapsulamento na Orientação a Objeto é que todo o objeto na sua estrutura elementar possui uma membrana de proteção ao longo do seu núcleo (ver Figura 5). O acesso ao estado e comportamento só é permitido através da passagem pela camada de comunicação representada pela membrana.

Em programação, somente a assinatura do método, ou seja, a chamada da função está publicada na interface. O conjunto da assinatura de todos os métodos também é conhecida como API (*Application Programming*

*Interface*). Para um desenvolvedor utilizar objetos é necessário que ele conheça somente API dos mesmos. A complexidade de como um Objeto foi implementado está encapsulada, não sendo relevante neste processo.

Pode-se ilustrar esta situação através da seguinte análise: quando se dirige um carro não é necessário compreender como o motor funciona com todas as suas 'milhares' de peças. O motorista abstrai a complexidade interna do motor e fixa-se na interface disponibilizada por quem construiu o carro, portanto, as interfaces seriam os pedais de aceleração, freio e embreagem, painel com medidor de velocidade etc.

A forma de garantir o encapsulamento de atributos e métodos é determinar a sua visibilidade privada através dos modificadores de acesso disponíveis nas linguagens de programação (*private* ou *protected*). Por exemplo, normalmente um atributo na linguagem java é declarado *private String nome*, sendo *private* uma palavra-chave indicando que a visibilidade do atributo *nome* só é permitida para métodos da própria classe.

A definição básica sobre a abstração é supressão de detalhe, visando controlar a complexidade pela ênfase em características essenciais. Na Orientação Objeto, pode-se aplicar este conceito no processo de especificação, construção e uso de classes.

Observa-se que os objetos identificados no mundo real normalmente são muito complexos. Por exemplo,

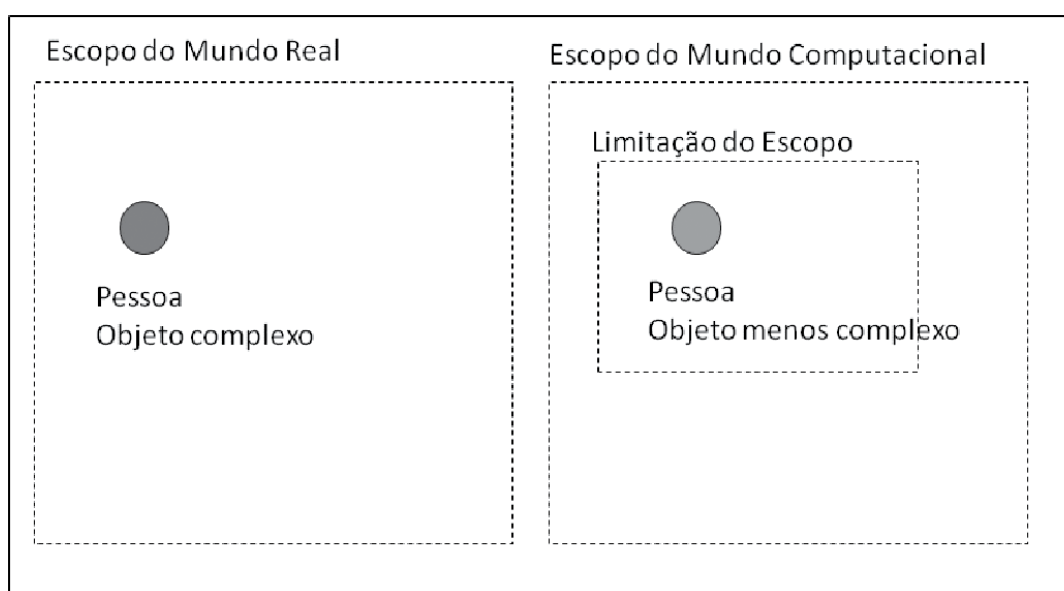


Figura 6 - Abstração de um objeto do mundo real  
Fonte: Os autores

imagine o que seria modelar uma classe Pessoa com todos os atributos e comportamentos!

Para a redução da complexidade da Classe Pessoa, é necessário delimitar o escopo com base no domínio do sistema que esta Pessoa será modelada, considerando somente os aspectos relevantes (ver Figura 6).

Por exemplo, para um sistema acadêmico, uma Pessoa pode representar um aluno, sendo nesse escopo somente necessário os atributos nome, matrícula e idade. As demais características que não são essenciais nesse contexto serão abstraídas. Como os detalhes ficam internos e escondidos no objeto Pessoa, pode-se subentender que este objeto possui quaisquer características de uma Pessoa do mundo real.

A abstração também se aplica na construção de uma classe. Uma primeira etapa é definir toda a interface da classe, ou seja, como os componentes externos irão interagir com a classe e qual o comportamento esperado pela mesma. O objetivo nesta etapa é somente de definir a interface (protocolo de comunicação), focando no que a classe deve fazer e não como a interface será implementada. Portanto, abstrai-se a implementação interna e concentra-se na especificação da interface que representa o protocolo de comunicação com o meio externo. Um bom critério de programação para a redução do acoplamento é programar para uma interface e não para uma implementação. Com a interface bem definida, pode-se abstrair o mundo externo e focar-se em como realizar o que foi especificado na interface.

## 2.3 HERANÇA DE CLASSE

Uma Classe pode representar genericamente uma ou mais Classes. Assim, Classes podem ser definidas a partir de uma Classe pai. O recurso de herança estabelece que uma Classe que herda de outra Classe possui todas as suas características (atributos e comportamentos).

O recurso de herança é semelhante à herança genética do mundo real. Logo, os filhos mantêm as características dos pais. Uma Classe pai, também denominada com superclasse, passa todas as suas características para a classe filha. A Classe filha ou subclasse, além de conter os atributos e comportamentos da superclasse, pode

acrescentar algo específico a sua funcionalidade ou reescrever métodos da superclasse. O termo *overridden* é utilizado quando a subclasse reescreve ou redefine um método herdado da superclasse.

Uma superclasse pode ser ainda definida como uma Classe concreta (como qualquer classe convencional) ou uma Classe Abstrata. Classes abstratas por definição não permitem a criação de objetos, portanto não é possível a criação de instâncias a partir destas Classes.

A definição de uma Classe como abstrata pode parecer algo estranho, visto que uma das funções principais de uma Classe é a geração de objetos. A declaração de Classes abstratas é algo comum e deve ser utilizado quando não faz sentido a geração de instâncias da superclasse.

Por exemplo, em uma estrutura hierárquica com a superclasse Pessoa e as subclasses PessoaFísica e PessoaJurídica, seria um erro a criação de instâncias a partir da classe Pessoa, já que existe uma disjunção entre as subclasses, ou seja, uma Pessoa ou é do tipo Física ou Jurídica.

Na classe abstrata, além de atributos e métodos, como uma classe concreta, têm-se os métodos abstratos. Métodos Abstratos não possuem implementação e são assinados na classe Abstrata. As classes que herdam da classe abstrata são obrigadas a implementar os métodos abstratos herdados da superclasse.

Associados ao conceito de herança, apresentam-se os conceitos de especialização e generalização. A especialização ocorre quando uma subclasse, além de herdar da superclasse, acrescenta características específicas diferentes da superclasse. Já o conceito de generalização ocorre quando existem Classes que possuem características em comum e cria-se uma Classe genérica, ou seja, uma superclasse que englobe essas características.

A Figura 5 apresenta duas subclasses PessoaFísica e PessoaJurídica que possuem atributos e um método em comum. Neste caso, aplica-se a generalização sobre os elementos em comum encapsulando-os em uma superclasse Pessoa que é uma abstração das subclasses.

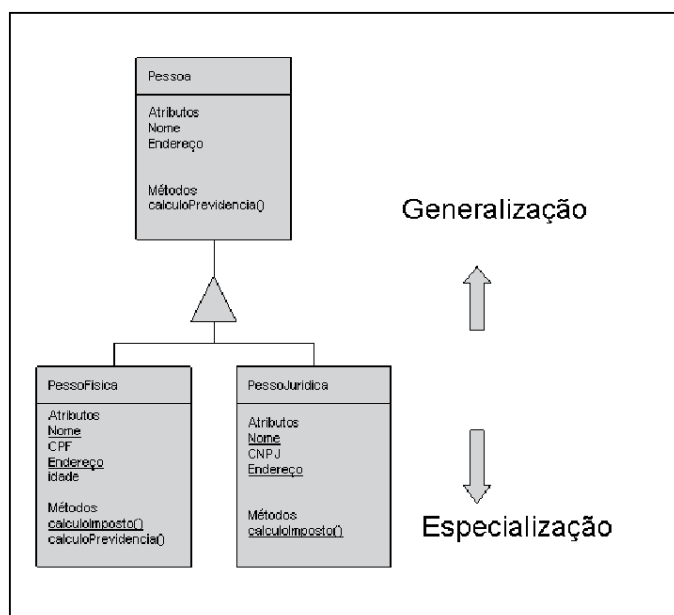


Figura 7 - Representação gráfica de Especialização e Generalização  
Fonte: Os autores

O uso de herança é um recurso poderoso, mas mal aplicado gera graves problemas nos projetos Orientados a Objetos. Muitos professores já apresentaram ou ainda apresentam a herança como o principal recurso da Orientação a Objetos e a forma ideal para alcançar a reutilização. Ao longo deste livro, irá se demonstrar que Orientação a Objetos vai muito além da herança e que a reutilização pode ser alcançada através de outras formas (ver seção sobre Herança versus Composição)

## 2.4 MÉTODOS, MÉTODO CONSTRUTOR E SOBRECARGA

A forma como os objetos colaboram em um sistema Orientado a Objetos é feita basicamente através da troca de mensagens. As mensagens são utilizadas para a comunicação entre objetos para a troca de dados e realização de ações. As ações são implementadas através de métodos que são publicados na camada de interface do objeto.

Em programação, os métodos são funções que possuem um parâmetro de retorno ou não, sendo, neste caso, do tipo void. Um método possui também um nome e seus parâmetros. Por exemplo, a figura 6 apresenta o método `public double getDistancia(int x, int y)` enumerando os seus elementos estruturais. O conjunto nome mais parâmetros também é conhecido como a

assinatura do método. Em uma classe não é possível a ocorrência de métodos com a mesma assinatura, porém podem existir métodos com o mesmo nome, mas parâmetros diferentes.



Figura 8 - Estrutura de Composição de um Método  
Fonte: Os autores

O método construtor é um método especial que está presente em todas as Classes. Este método deve possuir o mesmo nome da Classe. Por exemplo, a Classe Ponto poderia ter um método construtor `public Ponto()`. Além disso, diferente dos outros métodos, o método construtor não possui parâmetro de retorno, nem mesmo a declaração void.

Em uma classe podem existir vários métodos construtores, desde que possuam parâmetros diferentes. Outra característica importante é em relação a sua utilização. Quando um objeto nasce, ou seja, é instanciado a partir de uma Classe, o primeiro método a ser executado é o método construtor. Como foi mencionado, uma classe, obrigatoriamente, tem pelo menos um método construtor.

O conceito sobrecarga ou *overloading* é definido como métodos em uma classe que possui o mesmo nome, mas parâmetros diferentes. É importante ressaltar que pode-se ter vários métodos com o mesmo nome, pois o que identifica um método é o seu nome associado com os seus parâmetros.

Portanto, para que em uma Classe Ponto exista a ocorrência de sobrecarga de construtores, devemos ter, por exemplo, os métodos `public Ponto()` e `public Ponto(int x, int y)`. Observa-se que a ocorrência sobrecarga existe em métodos construtores ou não.

## 2.5 POLIMORFISMO

Objetos no mundo real que estão sobre uma mesma hierarquia podem possuir comportamentos definidos pelos seus pais, mas realizá-los de forma particular; por exemplo, uma superclasse FiguraGeométrica com

um método `public double calculoArea()`. Notoriamente, esta superclasse não tem como responder a este método, pois a função desta Classe é ser um conceito genérico. Logo, é uma Classe que tem o objetivo de definir como uma `FiguraGeométrica` deve funcionar genericamente.

O objetivo desta Classe é definir que toda a `FiguraGeométrica` tem o comportamento de cálculo de área. Já subclasses `Quadrado` e `Triângulo` vão possuir o método de cálculo de área, sendo que a implementação para cada Classe será feita de forma particular.

A importância de definir um conceito padronizado é a facilidade de absorver futuras mudanças. Por exemplo, uma nova classe `Retângulo` que herda de `FiguraGeométrica` tem também o método `public double calculoArea()`. Apesar de o método possuir a mesma assinatura, a sua realização seguirá as características da classe `Retângulo`. Conclui-se que um comportamento pode ter várias formas de ser respondido a depender do tipo de objeto. Este recurso é definido como **Polimórfico**.

A definição para o conceito **Polimorfismo** estabelece que, para um método genérico, pode-se especializá-lo e defini-lo em classes de tipos diferentes que responderão a este método de forma particular. O método genérico deve ser assinado em uma superclasse, determinando um padrão para as classes que irão herdar suas características. Diz-se assinado, pois o método da superclasse pode ser um método concreto ou abstrato. Normalmente, é um método abstrato.

A vantagem do **Polimorfismo** reflete-se no baixo acoplamento no projeto. Um objeto pode estar acoplado a um outro objeto sem necessariamente saber o seu tipo. Por exemplo, um objeto do tipo `Retângulo` pode ser manipulado como uma `FiguraGeometrica`. Caso o programa seja modificado para utilização de um `Triangulo`, ao invés de um `Retângulo`, o impacto com as mudanças no programa serão mínimas.

## 2.6 COESÃO

Para garantir a qualidade do projeto e código de um programa Orientado a Objetos, não basta apenas adotar conceitos tais como: Classe, Herança e Polimorfismo. É preciso garantir que cada objeto esteja assumindo o papel adequado e contribuindo da melhor forma para a finalidade do domínio do problema. Para tanto, os componentes desta unidade devem colaborar da melhor

forma.

Todo objeto no sistema deve ter uma finalidade bem definida. Representar um `Cliente`, um `Produto` ou uma `Empresa` são exemplos de responsabilidades que um objeto pode carregar. Definimos, por exemplo, que um objeto `Cliente` deve ter atributos como o nome, cpf, endereço e `limiteDeCRédito`.

A coesão avalia o quanto bem definido está a finalidade de um objeto e o quanto cada parte deste objeto colabora para atingi-la. Uma alta coesão indica que o objeto possui uma finalidade bem definida e os elementos deste objeto colaboram para tanto. Em contrapartida, uma baixa coesão indica que o objeto tem finalidades mal definidas ou em excesso ou ainda que existem partes do objeto que não colaboram da melhor forma.

Para evidenciar o que é coesão, sua importância e como atingir um alto nível, demonstram-se exemplos que desrespeitam a coesão, fazendo analogia a uma pequena empresa, onde os funcionários representam nossos objetos. Caso um funcionário trabalhe com ferramentas inadequadas, uma postura incorreta, um ambiente em que não há uma comunicação coerente entre seus colaboradores, certamente ele não desempenhará corretamente suas funções. Suas partes não estão colaborando com a finalidade do objeto funcionário, que, por sua vez, não irá conseguir colaborar plenamente com a finalidade da empresa (sistema). Caso suas partes (ferramentas, postura e ambiente) estejam colaborando perfeitamente, contudo sua função não seja bem definida, certamente este funcionário irá representar um desperdício de recursos e de tempo. Ele poderá ficar ocioso ou desempenhando uma função que lhe seja incompatível. Se atribuirmos a esse funcionário demasiadas funções, ocorrerá que ele ficará sobrecarregado, e não conseguirá desempenhar todas elas perfeitamente. Desta forma, ele não irá colaborar plenamente com a finalidade da empresa. Nesses casos estaríamos faltando com a coesão.

Devemos observar também que um altíssimo grau de coesão também é prejudicial para um projeto. Imagine se tivéssemos um funcionário para cada microfunção dentro da empresa. Seria um gasto desnecessário, e subestimariamos a capacidade deste funcionário. Portanto, devemos ponderar na mediada da coesão, garantindo que cada objeto colabore com a finalidade do sistema mantendo sua própria finalidade bem definida,

porém sem comprometer o desenvolvimento do projeto. É interessante também sempre deixar explícito (através de comentários e/ou documentação) os objetivos do objeto, principalmente quando este assumir responsabilidades complexas.



### SUGESTÃO DE ATIVIDADE

#### Questionário de Conceitos Básicos

1. Defina Objeto, explicando estado, comportamento e ciclo de vida.
2. Defina Classe, fazendo uma correlação com Objeto.
3. Explique os seguintes conceitos:
  - a. Encapsulamento
  - b. Sobrecarga
  - c. Método Construtor
  - d. Polimorfismo
4. Sobre herança de classe, explique os conceitos:
  - a. Classe Abstrata e Método Abstrato
  - b. Generalização/Especialização
  - c. Subclasse x Superclasse
  - d. Overridden (reescrever)
5. Qual a importância na definição primeiramente da interface da classe para depois efetuar a sua realização, ou seja, implementação do conteúdo interno da classe?
6. Qual a relação entre abstração e encapsulamento?
7. No projeto de uma classe, o que deve ser especificado e em que ordem?
8. Qual o objetivo, no projeto de uma classe, de defini-la como abstrata? E como seria o uso desta classe no contexto do programa OO?
9. Como você identificaria se um método é polimórfico?
10. O que define a assinatura de um método? E em que situação acontece a sobrecarga de métodos?
11. Explique como os objetos colaboram em um programa OO.
12. Sobre abstração, indique o V ou F. Caso F explique.

( ) É uma técnica para controlar a complexidade pela ênfase em características essenciais e pela supressão de detalhes.

( ) Para construção de modelos é importante desconsiderar alguns aspectos do mundo real, visando controlar a complexidade.

( ) Com a abstração vamos modelar somente o que é relevante para o contexto do sistema.

( ) Por exemplo, como os detalhes ficam internos e escondidos no objeto Pessoa, podemos entender que este objeto pode possuir qualquer característica de uma Pessoa do mundo real.

( ) No projeto de uma Classe não existe a necessidade do uso da técnica de abstração. Assim, devemos projetá-la com os seus atributos, métodos e interface.
13. Quais as consequências de um projeto que possui uma boa delegação de responsabilidades para suas unidades e seus objetos?



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

**APLICANDO OS  
CONCEITOS DE  
OBJETO, CLASSE,  
ENCAPSULAMENTO  
E SOBRECARGA NA  
LINGUAGEM JAVA**

# **CAPÍTULO 3**



## Anotações

Esta seção apresenta três exemplos simplificados na linguagem Java com o objetivo de ilustrar uma visão prática dos conceitos teóricos de OO. Portanto, este é o nosso primeiro contato com a linguagem Java.

Antes de iniciarmos a apresentação dos exemplos, determinaremos alguns padrões de nomenclatura apresentados no Quadro 1 para nome de classe, atributos, métodos, variáveis locais e constantes.



#### INDICAÇÃO DE LEITURA

O padrão aqui utilizado baseia-se na forma convencional de escrita de classes Java utilizada pela Oracle. Observa-se que este padrão é amplamente utilizado por desenvolvedores ao redor do mundo. Já, para entender mais sobre o padrão de codificação, acesse JAVA code conventions. Disponível em: <<http://www.oracle.com/technetwork/java/codeconventions-150003.pdf>>. Acesso em: 02 jul. 2013.

A definição de um padrão de nomenclatura é útil, pois ajuda nas futuras manutenções do código, normalmente efetuadas por pessoas diferentes.

Lembre-se de que o foco dos conceitos apresentados aqui não se resumem à construção de um bom algoritmo, mas o de criar um projeto robusto de fácil manutenção.

Somado a isto, o padrão de nomenclatura ajudará aos programadores iniciantes a compreenderem melhor as estruturas de programação. Como Java é uma linguagem case-sensitive, muitos erros são gerados por quem está iniciando nesta área, somente pela falta da adoção de um padrão de nomenclatura.

Agora que já conhecemos o padrão de escrita de um programa em Java, vamos iniciar os nossos estudos introdutórios de programação. Como dito na capítulo 2, antes da instanciação dos objetos, deve-se criar a classe. Uma classe Java é escrita em um arquivo fonte com o nome da classe e a extensão “.java”. Por exemplo, a classe de nome Ponto será escrita em um arquivo Ponto.java.

| Estruturas da Linguagem | Padrão de Nomenclatura (Regra geral é não colocar acento ou caracteres especiais nos nomes)                                                                                                                                                                                                                                                             |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nome da Classe          | Primeira letra maiúscula o resto do nome em Minúscula. Caso seja a composição de dois nomes, aplicar a regra anterior para o Segundo nome.<br>Exemplo:<br>Pessoa, PessoaFisica, PessoaJuridica                                                                                                                                                          |
| Atributos e variáveis   | A primeira parte do nome da variável deve ser em minúsculo. A segunda parte do nome, caso ocorra, deve possuir a primeira letra maiúscula e o resto do nome em minúscula.<br>Exemplo:<br>Nome, preco, qtdLitros                                                                                                                                         |
| Constantes              | Todas as letras do nome em maiúscula.<br>Exemplo:<br>ORDEM_NOME, ABERTO, VENCIDO                                                                                                                                                                                                                                                                        |
| Métodos                 | A primeira parte do nome do método deve ser em minúsculo. A segunda parte do nome, caso ocorra, deve possuir a primeira letra maiúscula e o resto do nome em minúscula. Uso do prefixo get e set para indicar métodos de recuperação de valores ou definição de valores respectivamente.<br>Exemplo:<br>getNome(), setNome(String nome), listaProduto() |

Quadro 1 - Padrões de nomenclatura  
Fonte: Os autores

Toda a classe em Java segue um padrão de formação na sua escrita. Na Figura 6, de forma simplificada, apresenta-se o padrão para a definição de uma classe em Java.

Nas próximas seções, apresentaremos três exemplos em Java para você entender como os conceitos de OO se aplicam nesta linguagem. Recomendo que você faça os

exemplos em um computador para entender melhor o funcionamento e possa alterar trechos de código para ter suas próprias experiências. Um questão importante também é saber ler os erros gerados pelo compilador do Java. Isto irá ajudar na solução da maioria dos problemas.

A classe inicia a sua declaração com: o modificador, depois a palavra reservada *class* e por fim o nome da classe.

```
public class Nome_Classe {
```

Exemplo de declaração da classe:

```
public class Ponto {
```

Normalmente a definição dos atributos é feita na primeira seção da classe, apesar disto não ser obrigatório. O modificador do atributo determinará a visibilidade do mesmo

Atributos

Modificador Tipo Nome\_Atributo;

...

Exemplo de declaração de um atributo:

```
private String nome;
```

Depois dos atributos são definidos os métodos da classe.

Modificador Tipo de Retorno Nome\_Método(Tipo Nome\_Parâmetro, Tipo Nome\_Parâmetro) {

Implementação do método

}

...

Exemplo de declaração de um método:

```
public int soma(int a, int b, int c){
```

```
    return a+b+c;
```

```
}
```

```
} – Fim da Classe
```

Figura 9 - Padrão de definição de uma classe em Java

Fonte: Os autores

### 3.1 EXEMPLO 1: PRIMEIRO PROGRAMA OO

Tem-se as classes *Ponto* e *DoisPontos*. A classe *Ponto* (utilizada como primeiro exemplo) é uma forma simplificada de escrita de uma classe, pois só possui atributos. Normalmente, classes possuem atributos e métodos. Na linha 5 da figura 10 da classe *Ponto*, são declarados os atributos *x* e *y*, também denominados como variáveis de instância. A classe *DoisPontos* também possui uma estrutura simplificada, visto que só será utilizada para criação de objetos. Esta classe não possui atributos e nem métodos de instancia. O método `public static void main(String args[])` é um método estático que será iniciado quando a classe *DoisPontos* for executada (ver linha 3 da figura 11). Os métodos estáticos são métodos que podem ser executados como funções da classe sem a necessidade de criação de instâncias. O método *main* é um método que existe em todas as classes que se deseja rodar o programa. Esta possibilidade pode gerar algumas dúvidas sobre qual classe deve representar o programa principal. Assim, deve-se definir uma classe que possui um método

*main*, semelhante à classe *DoisPontos*, que representa o programa principal. As outras classe que possuírem métodos *main* servirão como métodos de testes.

```
1 // Exemplo 1 (Classe Ponto)
2 // Início de Arquivo
3
4 public class Ponto {
5     int x,y;
6 }
```

Figura 10 - Listagem de código do Exemplo 1

Fonte do próprio Autor

As linhas 4 e 5 na classe *DoisPontos* utilizam o operador *new* para instanciação de dois objetos *Ponto*. Como a classe *Ponto* é uma fabrica de objetos pode-se ter a criação de *n* instâncias a partir desta classe. As variáveis *p1* e *p2* são do tipo *Ponto*. Em programação Orientada a Objetos, as classes são novos tipos definidos pelo desenvolvedor. A palavra *new Ponto()* indica a criação de um objeto da classe *Ponto*.

```

1 public class DoisPontos {
2
3 public void main(String args[]){
4     Ponto p1 = new Ponto();
5     Ponto p2 = new Ponto();
6     p1.x=10; p1.y=20; p2.x=43; p2.y=78; //Não desejável
7     System.out.println("Ponto 1-> x=" + p1.x + "y=" + p1.y); // Imprime
8     System.out.println("Ponto 2-> x=" + p2.x + "y=" + p2.y); // Imprime
9 }
10 }

```

Figura 11 - Listagem de código classe Dois Pontos

Fonte: Os autores

na memória. As variáveis p1 e p2 são do tipo Ponto, porque apontam para os endereços de memória que estão os objetos Pontos instanciados. As variáveis p1 e p2 podem acessar atributos e métodos públicos através do operador ponto (".") , por exemplo p1.x. O acesso direto de atributos é algo pouco recomendado, visto que fere a propriedade do encapsulamento (linha 6 da figura 11). O próximo exemplo ilustra como corrigir o problema de desrespeitar ou "furar" o encapsulamento. Um detalhe ainda, neste exemplo, é que em Java pode-se concatenar automaticamente um valor int com uma string (ver linha 7 figura 11).

### 3.2 EXEMPLO 2: EVOLUINDO O EXEMPLO 1 APLICANDO OS CONCEITOS DE ENCAPSULAMENTO

Este exemplo é um aprimoramento do exemplo 1. O problema de desrespeitar o encapsulamento, identificado no exemplo 1, é corrigido através da colocação do modificador privado na declaração dos atributos x e y (linha 3 da classe Ponto da figura 12). Um teste a ser feito, para verificar o bloqueio do acesso direto aos atributos, tenta executar a classe DoisPontos do exemplo anterior com a classe Ponto deste exemplo. Observe que não se consegue compilar a classe DoisPontos, pois é gerado um erro em tempo de compilação.

A principal diferença entre os dois exemplos é que, neste exemplo, o acesso aos atributos da classe Ponto será feito através de métodos de acesso get/set.

Outro detalhe diferente é a ocorrência da palavra reservada this (ver linha 5 da classe Ponto). A palavra this é utilizada internamente na classe para fazer referência a atributos e métodos da classe. Os atributos da classe são variáveis com escopo de visibilidade em qualquer método da classe que não seja estático. Assim, as variáveis que

```

1 package exemplo2;
2 public class Ponto {
3     private int x,y; //Dados encapsulados
4     public void setX(int x){
5         this.x=x; //this é um valor de referencia, que refere-se ao objeto corrente
6     }
7     public int getX(){
8         return this.x;
9     }
10    public void setY(int y){
11        this.y=y;
12    }
13    public int getY(){
14        return this.y;
15    }
16 }

```

Figura 12 - Listagem de Código do Exemplo 2

Fonte: Os autores

representam os atributos podem ser utilizadas (obtenção e alteração dos seus valores) em qualquer método.

Um erro comum entre os programadores iniciantes é quererem utilizar this no método main. Este é um erro grave, pois o método main não é um método de instância, e sim uma função da classe (um método estático - static), como mencionado no exemplo anterior.

### 3.3 EXEMPLO 3: EXEMPLIFICANDO MÉTODO CONSTRUTOR E SOBRECARGA

Este exemplo apresenta dois métodos importantes: o método construtor e o método toString. O método construtor é um método especial e diferente, pois não possui nenhum retorno (nem mesmo void). O método construtor deve ter o mesmo nome da classe, e será o primeiro método a ser chamado quando o objeto for instanciado (ver linha 24 figura 13)

```

1 package exemplo3;
2 public class Ponto {
3     private int x,y; //Dados encapsulados
4     public Ponto(int x, int y){
5         this.x=x;
6         this.y=y;
7     }
8     public Ponto(int x){
9         this(x,40); //O atributo y é fixo em 40. Esta linha fará a chamada ao construtor
10        //que recebe dois parametros.
11    }
12    public Ponto(){
13        this(-1); //O atributo x e y são fixos em -1 e 40 respectivamente.
14    }
15    public String toString(){
16        return "Ponto X=" + this.x + "Y=" + this.y;
17    }
18    public static void main(String args[]){
19        Ponto p=new Ponto (10,20);
20        System.out.println(p);
21        Ponto p1=new Ponto (10);
22        System.out.println(p1);
23        Ponto p2=new Ponto ();
24        System.out.println(p2);
25    }
26 }

```

Figura 13 - Listagem de código Exemplo 3

Fonte: Os autores

Em uma classe podem ser definidos n métodos construtores, contanto que estes possuam parâmetros diferentes (conceito de sobrecarga), ver as linhas 19 até 24 da figura 13. Em Java, caso não seja definido um método construtor por padrão, tem-se um construtor vazio (situação apresentada no dois exemplos anteriores). A palavra reservada *this*, apresentada no exemplo anterior, está sendo usada para a reutilização e a chamada de um método construtor por outro método construtor (ver as linhas 4 até 14 da figura 13). O método *toString* é um método padronizado em java para informar normalmente o estado do objeto. Assim, quando se imprime a referência de um objeto, automaticamente o método *toString* é chamado (ver linha 25 da figura 13). O comando *System.out.println(p)* é o mesmo que *System.out.println(p.toString())*. (faça o teste)

O método *toString()* e outros métodos como equals estão definidos em uma classe *Object*. Esta classe é a classe pai de todas as classes em java. Assim, qualquer classe que seja criada em java por padrão herda da classe *Object* (sem a necessidade de nenhuma diretiva como *extends* que veremos no próximo exemplo). Um teste a ser feito para a comprovação disto é comentar o método *toString()* da classe *Ponto* e executar - lá. Observe que o programa funciona apesar da impressão ser algo inesperada. O método *toString()* da classe *Object*, herdado pela classe *Ponto*, redefiniu o método *toString()*, mantendo a mesma assinatura, mas mudando a sua implementação (overridden).

### 3.4 HERANÇA, POLIMORFISMO E CAST

Nesta seção, vamos abordar sobre o uso da Herança, Polimorfismo e Cast. Estes são conceitos poderosos, favorecem a abstração e se complementam. A seguir, explicamos e exemplificamos cada um deles.

#### 3.4.1 Exemplo 4: Exemplificando Herança

Este exemplo mostra como pode ser implementado o conceito de herança em Java. Na figura 14, é exibida a classe *Ponto3D* que é uma especialização da classe-pai (superclasse) *Ponto* (figura 9).

Na linha 2 da figura 14, nota-se a presença da palavra reservada *extends* do Java para indicar que a classe

*Ponto3D* herda da classe *Ponto*.

Dessa forma, um objeto do tipo *Ponto3D* tem todas as características da classe *Ponto*, e ainda sua especialização, que corresponde a 3ª dimensão (Z). Pode ser observado ainda que, na linha 6, tem-se a palavra *super*, reservada do Java, utilizada para fazer referência à super classe que, no exemplo *super(x,y)*, está fazendo referência ao construtor da classe *Ponto* (*int x\_,int y\_*).

```

1 package exemplo5;
2 public class Ponto3D extends Ponto {
3     int z; // Dados encapsulados
4
5     Ponto3D(int x, int y, int z) {
6         super(x,y);
7         this.z=z;
8     }
9
10
11     void setZ(int z){
12         this.z = z; // this é um valor de referência
13         //que refere-se ao objeto corrente
14     }
15
16     int getZ(){
17         return this.z;
18     }
19
20
21     public String toString(){
22         return "Ponto3D X=" + this.getX() +
23             " Y="+this.getY()+" Z="+this.getZ();
24     }
25 }

```

Figura 14 - Listagem de código Exemplo 5

Fonte: Os autores

A figura 14 mostra um exemplo de como utilizar a estrutura de herança definida para *Ponto* e *Ponto3D*. Na linha 6 da figura 14, é definido um objeto do tipo *Ponto* e logo em seguida, na linha 7, o objeto do tipo *Ponto* é instanciado do tipo da subclasse *Ponto3D*. Nesse caso, para acessar métodos que existem na superclasse, eles deverão ser invocados normalmente como é feito na linha 9 – *getX()*. Entretanto, caso se deseje acessar atributos da subclasse *Ponto3D*, deverá ser feita uma conversão explícita de objetos que denomina-se de CAST para a classe filha *Ponto3D*, como mostrado na linha 10.

```

1 package exemplo5;
2
3 public class Principal {
4
5     public static void main(String args[]){
6         Ponto p;
7         p = new Ponto3D(10, 20,30);
8         System.out.println(p);
9         System.out.println(p.getX());
10        System.out.println("Z =" + (Ponto3D) p).getZ() );//Cast
11    }
12 }
13 }

```

Figura 15 - Listagem de código Exemplo 5  
Fonte: Os autores

### 3.4.2 Exemplo 5: Exemplificando Herança - 2

A figura 15 apresenta a classe Horário que corresponde à abstração do horário em um determinado ponto na linha do tempo. A classe Horário possui o método construtor `public Horário(int hora,int minuto)` e também o método `public int tempoTranscorrido()` que retorna o tempo em minutos.

```

1 package exemplo8;
2 public class Horário{
3     protected int hora,minuto;
4
5     public Horário(int hora, int minuto){
6         this.hora = hora;
7         this.minuto = minuto;
8     }
9
10    public int getHora(){
11        return this.hora;
12    }
13
14    public int getMinuto(){
15        return this.minuto;
16    }
17
18    public int tempoTranscorrido(){
19        return this.hora*60 + this.minuto;
20    }
21 }

```

Figura 16 - Listagem de código Exemplo 8  
Fonte: Os autores

Na figura 16, temos um exemplo de herança, pois a classe HorárioS estende de Horário, herdando atributos e comportamentos da classe Horário. Na classe HorárioS e incluindo os seus próprios, podendo, como já foi abordado, reescrever métodos da superclasse, *overridden*. Na figura 16, mostra que a subclasse HorárioS reescreve o método `public int tempoTranscorrido()` da

superclasse Horário, para retornar o tempo em segundos.

Na linha 5, encontra-se a invocação do construtor da superclasse. Como já abordado, `super` é uma palavra-chave utilizada para invocar explicitamente atributos (esquecendo-se o encapsulamento) ou métodos da superclasse.

Salientando que torna-se necessário, para invocar o construtor do superclasse, que esse comando seja o primeiro do construtor da subclasse. Na linha 10, temos outro exemplo da utilização desta palavra-chave.

```

1 package exemplo8;
2 public class HorárioS extends Horário {
3     private int segundo;
4
5     public HorárioS(int newHora, int newMinuto, int newSegundo){
6         super(newHora,newMinuto);
7         this.segundo = newSegundo;
8     }
9
10    public int tempoTranscorrido(){
11        return super.hora*60*60 + super.minuto*60 + this.segundo;
12    }
13
14 }

```

Figura 17- Listagem de código Exemplo 8  
Fonte: Os autores

Na figura 17, nota-se um exemplo de como utilizar a herança definida entre a classe Horário e a classe HorárioS. Na linha 6 e 7, são definidas duas variáveis `h1` e `h2` do tipo Horário e instanciadas. Na linha 8, são invocados os métodos tempo transcorridos dos dois objetos, `h1` e `h2`, e calculada a diferença em minutos entre eles. Nas linhas 11 e 12, são definidos dois objetos do tipo Horário, `h3` e `h4`, entretanto instanciado a partir da subclasse HorárioS, o que é possível pois HorárioS é uma subclasse de Horário. Sendo feito assim (sem uso explícito), ao serem invocados métodos nesses objetos, serão acessados os métodos que existem na classe Horário. Para o caso de *overridden*, especialmente, será invocado o método da subclasse, devido a instância do objeto. Já, para se ter acesso a um método específico da subclasse, torna-se necessário fazer um explícito para o tipo HorárioS. No exemplo para se invocar os segundos definidos em HorárioS, será necessário invocar o método `getSegundos()`.

```

1 package exemplo8;
2
3 public class Principal {
4
5     public static void main(String args[]){
6         Horario h1 = new Horario(12,30);
7         Horario h2 = new Horario(11,40);
8         int diferenca = h1.tempoTranscorrido() - h2.tempoTranscorrido();
9         System.out.println(diferenca);
10
11         Horario h3 = new Horario(12,30,30);
12         Horario h4 = new Horario(11,40,30);
13         diferenca = h3.tempoTranscorrido() - h4.tempoTranscorrido();
14         System.out.println(diferenca);
15     }
16 }

```

Figura 18 - Listagem de código Exemplo 8  
Fonte: Os autores

### 3.4.3 Exemplo 6: Exemplificando Classe Abstrata e Polimorfismo

Nesse exemplo, é mostrado herança utilizando classe abstrata e polimorfismo. Na figura 18, temos a classe abstrata Letra, que será a superclasse do exemplo. Percebe-se que ela possui o método abstract String Imprime() que devera ser, obrigatoriamente, implementado pelas subclasses.

Na linha 6 da figura 18, apresenta a assinatura do método estático (permite ser acessado sem instanciar um objeto do tipo da classe, o que é impossível instanciar para o exemplo fornecido, pois é uma classe abstrata) imprimeLetra que recebe como parâmetro um objeto do tipo Letra e imprime no console o seu conteúdo.

```

1 package exemplo6;
2 public abstract class Letra {
3
4     abstract String Imprime();
5
6     public static void imprimeLetra(Letra l){
7         System.out.println(l.Imprime());
8     }
9 }

```

Figura 19 - Listagem de código Exemplo 6  
Fonte: Os autores

Na figura 19, tem-se a subclasse Consoante que herda da classe Letra. Como já dito, toda subclasse de Letra tem que implementar o método abstrato Imprime(), o que é feito na figura mencionada.

```

1 package exemplo6;
2 public class Consoante extends Letra{
3
4     public String Imprime(){
5         return "Consoante";
6     }
7
8 }

```

Figura 20 - Listagem de código Exemplo 6  
Fonte: Os autores

Na figura 20, tem-se a subclasse Vogal que também implementa o método Imprime().

```

1 package exemplo6;
2 public class Vogal extends Letra{
3
4     public String Imprime(){
5         return "Vogal";
6     }
7
8 }

```

Figura 21 - Listagem de código Exemplo 6  
Fonte: Os autores

Na figura 21, nas linha 7 e 8, são instanciados (criados) dois objetos, c e v, como Consoante e Vogal respectivamente. Na linha 10 e 11, é invocado o método estático imprimeLetra da classe Letra passando como parâmetro os objetos instanciados. Na linha 10, será retornado "Consoante" e, na linha 11, será retornado "Vogal", que corresponde a um exemplo de polimorfismo o qual, a depender do objeto, será invocado o mesmo método que retornara um valor diferente da dependente do objeto passado como parâmetro

```

1 package exemplo6;
2
3 public class Principal {
4
5     public static void main(String args[]){
6
7         Letra c = new Consoante();
8         Letra v = new Vogal();
9
10        Letra.imprimeLetra(c);
11        Letra.imprimeLetra(v);
12
13    }
14 }
15 }

```

Figura 22 - Listagem de código Exemplo 6

Fonte: Os autores



## SUGESTÃO DE ATIVIDADE

## Questionário Aplicando OO na Linguagem Java

Indique a sequência de etapas que você seguiria para a especificação e construção de um programa orientado a objetos até a sua execução.

2) Indique as falhas na implementação da classe Ponto escrita a baixo.

```

1     public abstract class Ponto {
2         public int x, y;
3
4         public Ponto(int x, int y) {
5             this.x = x;
6             this.y = y;
7         }
8         public Ponto() {
9             super(1, 3);
10        }
11        public String toString() {
12            return "Ponto X= " + this.x + " Y=
13            " + this.y;
14        }
15        public static main(String args[]) {
16            this.x = 100;
17            this.y = 200;
18            Ponto p = new Ponto(20, 30, 10);
19        }

```

3) Considere a classe a seguir cuja instância (objeto) representa um Login(nome, senha). Seu método verificaLogin retorna se um nome e senha passados como parâmetros são iguais ao estado do objeto.

```

1     public class Login {
2         protected String nome;
3         protected String senha;
4         public Login(String nome, String senha) {
5             this.nome = nome;
6             this.senha = senha;
7         }
8         public String getNome() {
9             return this.nome;
10        }
11        public String getSenha() {
12            return this.senha;
13        }
14        public boolean verificaLogin(String nome,
15        String senha) {
16            return ((this.nome.equals(nome)&&this.senha.
17            equals(senha)) ?
18                true : false);
19        }
20    }

```

Baseando-se na classe a cima, implemente um método **main** que crie uma instância da classe Login com o seguinte estado: nome = eduardo e senha = 123. Em seguida, imprima o valor de retorno do método verificaLogin com os parâmetros nome = carlos e senha = 123.

Obs: comando para impressão System.out.println();

4) Crie uma classe Pessoa com os atributos *String nome* e *int idade*, métodos *Get* para os seus atributos, o método construtor *public Pessoa(String nome\_, int idade\_)*, um segundo método construtor *public Pessoa(String nome\_)* que reutilize o método construtor anterior fixando o valor da idade em 20, e implemente também o método *public String toString()*. Em seguida, crie um método **main**, instancie dois objetos Pessoa, utilizando o primeiro e o segundo método construtor, e imprima seus valores; para o primeiro objeto utilize seu toString implicitamente, para o segundo utilize os métodos Get.



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

**VANTAGENS  
E DIFERENÇAS  
ENTRE A  
PROGRAMAÇÃO  
FUNCIONAL E A  
PROGRAMAÇÃO  
ORIENTADA A OBJETOS**

# **CAPÍTULO 4**



## Anotações

A programação F. Estruturada e a programação Orientada a Objetos estão em paradigmas bem diferentes. Por isso, os primeiros passos na Orientação a Objetos são, às vezes, tão complicados. Na programação F. Estruturada, o desenvolvedor possui uma liberdade maior na escrita do seu código, sendo mais fácil à criação de algoritmos pontuais, por exemplo, a construção de algoritmos de ordenação ou que implementam funções matemáticas. Os programas no paradigma F. Estruturada possuem um fluxo principal que é organizado de forma modular através de um conjunto de funções que podem ser chamadas ao longo do fluxo ou por outras funções.

A programação Orientada a Objetos não foi projetada para substituir a programação F. Estruturada. Essas duas formas já convivem por muitos anos e, certamente, ainda coexistirão ao longo do tempo. Muitos dos conceitos apresentados na programação F. Estruturada são aplicados na programação Orientada a Objetos. Toda a estrutura de linguagem como os comandos de loop (while, for), condicionais (if, operações condicionais), variáveis locais entre outros são mantidos na programação OO. Funções e a técnica de modularização características marcante da programação F. Estruturada também estão presentes na programação Orientada a Objetos, agora com um novo enfoque. Assim, pode-se afirmar que a programação Orientada a Objetos é, na verdade, uma evolução da programação F. Estruturada, visando solucionar problemas mais complexos.

Muitos projetos de software foram e continuarão sendo construídos na programação F. Estruturada. Todavia, existem algumas dificuldades notórias neste modelo. Uma das principais dificuldades encontra-se no contexto da modelagem de sistemas. A modelagem do mundo real e a transformação deste modelo em um programa de computador no paradigma F. Estruturado não é um processo intuitivo, além de não existir um esquema formal que apoie esse processo. Observa-se que a notação para construção de um projeto de software no paradigma F. Estruturado não tem uma correlação direta entre os conceitos modelados e os algoritmos a serem construídos. É importante ressaltar que o mundo real é composto por entidade e não por funções. Além da problemática supracitada, questões como

a baixa reutilização de código (veremos um exemplo prático sobre esta questão), dificuldade na delegação semântica das responsabilidades dentro do programa, ou seja, qual parte do programa deve ser responsável por conter determinada função e códigos, muitas vezes com alto grau de acoplamento, estão presentes na programação F. Estruturada. Conclui-se, então, a existência de problemas tanto na modelagem como na construção de projetos de software e dificuldades na manutenção, seja esta evolutiva ou corretiva. Não existe uma forma de fazer uma correlação direta entre a Programação F. Estruturada e a Programação Orientada a Objetos. Subtendendo a aproximação dos conceitos, verifica-se que os dados utilizados nas funções podem ser representados pelo estado do objeto e que as funções são os métodos do objeto (ver Figura 22). Ressalta-se que a principal diferença entre esses dois paradigmas consiste no aspecto que, na programação F. Estruturada, os dados e funções não estão ligados diretamente, dificultando a reutilização das funções e semântica da modelagem do mundo real. Já na orientação a objetos, os dados e funções estão estritamente relacionados, sendo conjugados através de um módulo semântico representado pelo objeto. Outro ponto importante a ser ressaltado é que os dados referentes ao objeto estão armazenados no próprio objeto, mantendo o seu estado ao longo do seu ciclo de vida.

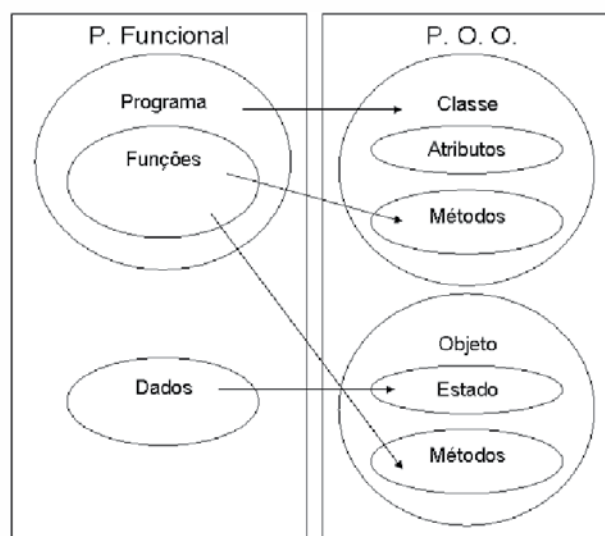


Figura 23 - Correlação entre a Programação F. Estruturada e a Programação Orientada a Objetos

Fonte: Os autores

#### 4.1 BENEFÍCIOS ALCANÇADOS COM A OO: ACOPLAMENTO E COMPLEXIDADE, QUANTO MENOR MELHOR

A meta de um desenvolvedor no projeto e implementação de um software deve ser sempre obter o máximo de redução de acoplamento e complexidade. É dito redução, pois um software sempre terá algum nível de acoplamento, visto os objetos estarem relacionados e trocarem mensagens entre si. Um projeto de software Orientado a Objeto que respeita a propriedade do encapsulamento, mesmo não usando nenhuma outra técnica (como padrões de projeto que será abordado no capítulo X), já está em um bom caminho para alcançar a redução de acoplamento e complexidade.

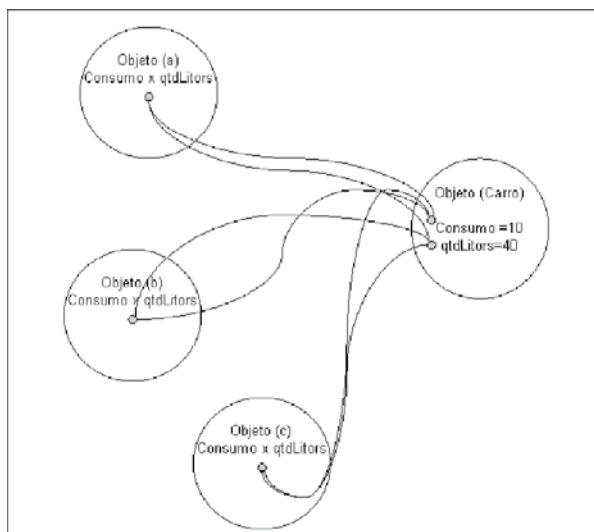


Figura 24 - Exemplo de implementação como forte acoplamento  
Fonte: Os autores

Portanto, uma vantagem básica da encapsulação é facilitar mudanças isolando pedaços uns dos outros (reduzindo o acoplamento). Para melhor entendimento de como o encapsulamento pode beneficiar a redução do acoplamento, ilustra-se com um exemplo prático.

No contexto do cálculo da autonomia de um carro, supondo que exista uma regra de negócio mapeada pela fórmula  $R(1) = (\text{consumo} * \text{qtdLitros})$ . Neste exemplo, têm-se três objetos a, b e c que necessitam da informação da autonomia. Para isso, esses três objetos acessam um quarto objeto do tipo Carro, obtendo seus dados de consumo e qtdLitros. Como ilustrado na figura 23, o algoritmo para o cálculo da autonomia está

implementado nos objetos a, b, e c. Este exemplo está totalmente desalinhado como os conceitos da Orientação a Objetos, pois desrespeita o encapsulamento, visto que o cálculo da autonomia é um comportamento que deve estar encapsulado no objeto Carro.

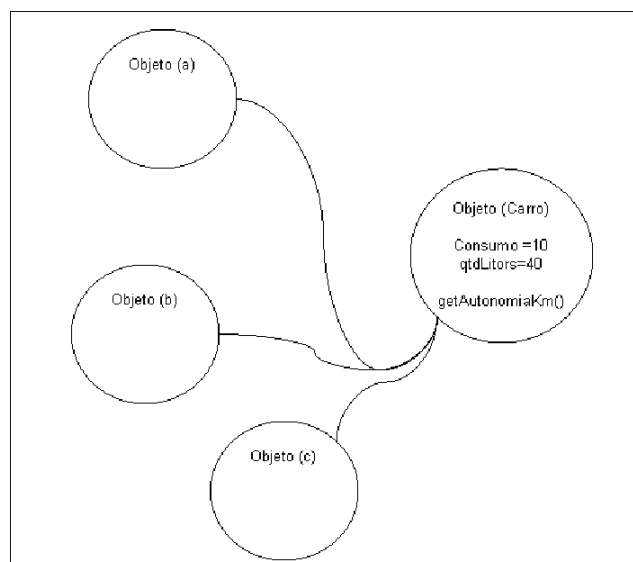


Figura 25 - Exemplo de implementação com um fraco acoplamento  
Fonte: Os autores

O acoplamento entre os objetos pode fazer com que uma mudança tenha repercussão em vários pontos do software, dificultando o processo de manutenção. Para o exemplo anterior, suponha, agora, que haja uma alteração na regra de negócio referente à autonomia, e a fórmula seja mudada para  $R(1) = (\text{consumo} * \text{qtdLitros} * \text{percentualPerda})$ . O resultado desta simples mudança tem impacto significativo; é uma alteração nos objetos a, b, c.. A forma correta para esta situação é encapsular o algoritmo referente à regra de negócio  $R(1)$  no objeto Carro e disponibilizar, na sua interface, um método double `getAutonomiaKm()` como apresentado na Figura 24. Assim, caso haja alguma mudança na fórmula da autonomia do Carro, esta fica restrita ao objeto Carro.

Outro critério que o desenvolvedor deve buscar ao longo do projeto de um software é a redução da complexidade do mesmo. A técnica de modularização, amplamente utilizada no paradigma funcional, estabelece que, para problemas complexos, deve-se dividi-los em partes menores, criando unidades funcionais menores de mais fácil compreensão e desenvolvimento. Caso seja aplicável devido ao tamanho de um projeto, deve-se decompor o sistema em subsistemas menores para

reduzir a complexidade da situação original. Ao se quebrar o problema em partes menores e mais simples, é possível identificar e estabelecer as camadas e os módulos do sistema, e suas interdependências. A divisão em camadas é uma estrutura desejável, pois reduz sensivelmente o acoplamento dos subsistemas, visto que, uma camada é um subsistema que interage apenas com o subsistema imediatamente inferior ou superior. Os módulos (chamados de pacotes, em Java) são unidades menores que compõem os subsistemas e agrupam as unidades funcionais – as classes. Um critério que deve ser adotado para encapsular classes em um módulo é que as classes que pertencem a um mesmo módulo devem ter um maior nível de acoplamento entre si, ao contrário das classes pertencentes a módulos distintos, que possuem um baixo nível de acoplamento.



#### SUGESTÃO DE ATIVIDADE

**Vamos, neste ponto, fazer uma atividade de revisão condensando pontos de assuntos que já foram apresentados e a redução de acoplamento e complexidade vistos nesta seção. Bons estudos**

##### Questão 1

Marque a alternativa correta sobre as definições de Objeto e Classe.

- a. Objeto é uma estrutura para representar elementos do mundo real composta por estado, comportamento, ciclo de vida e identidade
- b. Classe é composta por estado e comportamento
- c. Todas estão corretas
- d. Classe é uma forma de tipificar Objetos do mesmo tipo e é criada com base em um Objeto
- e. Classe é uma estrutura dinâmica utilizada para criar Objetos

##### Questão 2

Complete a frase corretamente: A encapsulação facilita as mudanças.....

- a. pois aumenta a velocidade do software.
- b. pois isola pedaços reduzindo complexidade e acoplamento.
- c. pois garante que o software foi feito corretamente.
- d. pois isola pedaços garantido que não existe complexidade e acoplamento.

##### Questão 3

Sobre encapsulamento marque a alternativa correta.

- a. É uma forma de comunicação entre objetos.
- b. É uma forma de isolar partes de um software.
- c. É uma forma de proteger dados.
- d. É uma forma de proteger comportamentos.

##### Questão 4

Sobre Generalização e Especialização, marque a alternativa correta.

- a. O que existe de comum em duas Subclasses pode ser especializado e empacotado na Superclasse.
- b. Para existir a especialização não é necessário ocorrer generalização em uma estrutura hierárquica de Classes.
- c. Especialização é o que existe de diferente em um Subclasse em relação a sua Superclasse.
- d. Pode ocorrer quando não existe uma estrutura hierárquica de Classes.

##### Questão 5

Sobre Classe Abstrata, Polimorfismo marque a alternativa correta

- a. Polimorfismo só existe quando temos duas classes com uma Superclasse Abstrata ou não.
- b. Polimorfismo só existe quando temos três classes com uma Superclasse Abstrata.
- c. Polimorfismo só existe quando temos três classes com uma Superclasse Abstrata ou não e obrigatoriamente um método abstrato assinado na Superclasse.
- d. Polimorfismo só existe quando temos três classes com uma Superclasse Abstrata ou não e obrigatoriamente um método assinado na Superclasse Abstrata ou não.
- e. Nenhuma alternativa está correta.

##### Questão 6

Marque a alternativa incorreta.

- a. Uma classe Abstrata só tem atributos e métodos abstratos.
- b. Classe Abstrata não pode criar instâncias.
- c. Método abstrato só existe em Classe Abstrata.
- d. Classe Abstrata permite definir um tipo genérico.
- e. Classe Abstrata é sempre uma Superclasse.

##### Questão 7

Marque a alternativa correta sobre o conceito Sobrecarga.

- a. Métodos iguais em Classes diferentes.
- b. Classes da mesma hierarquia com métodos de nomes diferentes.
- c. Métodos com parâmetros iguais e o nome diferente.
- d. Na mesma Classe, métodos com mesmo nome, mas com parâmetros diferentes.

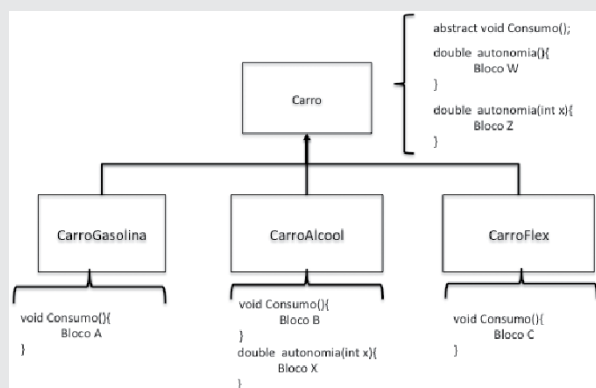
##### Questão 8

Escreva sobre as principais diferenças entre a Programação OO versus a Programação Funcional.

##### Questão 9

Explique abstração apresentado as possibilidades de aplicação em um projeto OO.

Responda as próximas questões com base na figura:



Questão 10

Marque a alternativa correta para os métodos na Classe CarroAlcool com os seguintes recursos, respectivamente, polimorfismo, sobrecarga e overriden (sobrescrever)

- Consumo(), autonomia() e Consumo()
- autonomia(), autonomia() e Consumo()
- autonomia(), autonomia() e autonomia(int x)
- Consumo(), autonomia() e autonomia(int x)
- Todas estão corretas.

Questão 11

Marque a alternativa incorreta.

- Existe Polimorfismo em dois métodos.
- Existe sobrecarga em mais de uma Classe.
- Existe sobrecarga em uma subclasse.
- Existe Overriden (sobrescrever) em mais de um método.
- Existe classe Abstrata e Método Abstrato.

Questão 12

Marque V ou F (uma alternativa errada zera a questão).

| N  | Comando                             | Vou F |
|----|-------------------------------------|-------|
| 1  | Carro c = new CarroGasolina();      |       |
| 2  | Carro c1 = new CarroAlcool();       |       |
| 3  | CarroGasolina c2 = new CarroFlex(); |       |
| 4  | CarroFlex c3 = new CarroFlex();     |       |
| 5  | CarroFlex c4 = new CarroFlex(1);    |       |
| 6  | c.autonomia(10);                    |       |
| 7  | c1.Consumo();                       |       |
| 8  | c3. autonomia(20);                  |       |
| 9  | c4.autonomia();                     |       |
| 10 | ((CarroGasolina) c).autonomia(10);  |       |

## 4.2 EXEMPLO PRÁTICO EM JAVA DE COMPARATIVO DE PROGRAMAÇÃO OO VERSUS PROGRAMAÇÃO F. ESTRUTURADA

Esta seção será norteadada por um problema para que, através da solução, você faça uma compreensão ampliada da Programação OO Versus Programação F. Estruturada. Suponha o seguinte problema:

Um algoritmo para calcular a autonomia do carro (função Km) e para calcular quanto esse carro pode rodar antes de atingir o combustível que está no tanque de reserva (função KmParaReserva) de um Carro. Suponha que o tanque desse Carro é de 8 litros e o consumo e a quantidade de combustível no tanque são variáveis de entrada.

Exemplo:

Informações de Entrada do Programa

- Placa: JPK 3306
- Consumo: 10 Km por Litro
- Qtd Litros: 40 Litros

Informações de Saída do Programa

- Carro Placa: JPK3306
- Km: 400 Km
- KmParaReserva: 320 Km

A Figura 25 apresenta a solução do problema usando as técnicas de Orientação a Objetos e a Figura 26 aplicando os conceitos do paradigma F. Estruturado. A classe Console é uma classe utilitária. O seu código está descrito no Anexo I.

As próximas seções apresentam um programa implementado nos dois paradigmas.

```
1.      Exemplo (Classe Carro)
2. public class Carro {
3.
4. // Declaração dos Atributos
5.     private String placa;
6.     private double consumo;
7.     private double qtdLitros;
8.
9. // Método Construtor
9. public Carro( String placa_, double consumo_, double qtdLitros_){
10.
11. // Definição dos valores dos atributos através dos parâmetros
11.     this.placa = placa_;
12.     this.qtdLitros = qtdLitros_;
13.     this.consumo = consumo_;
14. }
15.
16. // Método que retorna a autonomia do Carro
16. public double getKm(){
17.
17.     return this.consumo*qtdLitros;
18. }
19. // Método que retorna a autonomia Carro antes de entrar na reserva
20. public double getKmParaReserva(){
21.
21.     return this.getKm()/8*consumo;
22. }
23.
24. // Método que retorna o valor referente a Placa do Carro
24. public String getPlaca(){
25.
25.     return this.placa;
26. }
27. // Método que será executado quando a classe for rodada
28. public static void main(String[] args) {
29.
29.     while (true){
30.
30.         //Obtém a placa do teclado
31.         String placa = Console.readString("Indique a Placa do Carro:");
32.         //Obtém o consumo do teclado
33.         double consumo = Console.readDouble("Indique o consumo do Carro:");
34.         //Obtém a quantidade de litros no tanque do teclado
35.         double qtdLitros = Console.readDouble("Indique a Quantidade de Litros do Carro:");
36.         // Criação de uma instância de carro
37.         Carro carro = new Carro(placa,consumo,qtdLitros);
38.         //Impressão da Placa do Carro
39.         System.out.println("Carro Placa: "+carro.getPlaca());
40.         // Impressão do Autonomia e da Autonomia para atingir a reserva
41.         System.out.println(" KM: " + carro.getKm()+
42.         " KM para Reserva: " + carro.getKmParaReserva());
43.     }
44.
45. }
46. }
```

Figura 26 - Exemplo da Solução do Algoritmo na Programação Orientada a Objetos

Fonte: Os autores

```
1. Exemplo (Classe CarroF)
2. class CarroF {
3.     //Função que retorna a autonomia do Carro
4.     public static double getKm(double consumo, double qtdLitros){
5.
6.         return consumo*qtdLitros;
7.
8.     //Função que retorna a autonomia do Carro antes de entrar na reserva
9.     public static double getKmReserva(double consumo, double qtdLitros){
10.
11.         return CarroF.getKm(consumo,qtdLitros) - 8*qtdLitros;
12.     }
13.
14.     // Função executado quanto a classe for rodada
15.     public static void main(String args[]) {
16.
17.         while (true){
18.
19.             String placa = Console.readString("Indique a Placa do Carro:");
20.             double consumo = Console.readDouble("Indique o consumo do Carro:");
21.             double qtdLitros = Console.readDouble("Indique a Quantidade de Litros do Carro:");
22.
23.             System.out.println("Carro Placa:"+placa);
24.
25.             System.out.println(" Km: " + CarroF.getKm(consumo,qtdLitros)+
26.             " Km para Reserva: " + CarroF.getKmReserva(consumo,qtdLitros) );
27.
28.         }
29.     }
30. }
```

Figura 27 - Exemplo da Solução do Algoritmo na Programação F. Estruturada  
Fonte: Os autores

A conclusão aferida sobre estes dois paradigmas é que são complementares e que o paradigma Orientado a Objetos é uma evolução do paradigma F. Estruturado. No processo de assimilação, pode-se afirmar que o início do aprendizado na Orientação a Objetos possui uma curva de aprendizado bem maior, visto que se tem uma quebra de paradigma na forma convencional de programação e especificação. Além disso, existe o envolvimento de um maior número de definições e conceitos. Em contrapartida, uma vez feita a assimilação dos conceitos sobre Orientação a Objetos certamente será difícil pensar ou projetar/implementar software de outra forma.



## SUGESTÃO DE ATIVIDADE

**Questionário de Comparativo OO e Programação Funcional**

Vamos, neste ponto, fazer mais uma atividade de revisão juntado com o assunto anteriormente apresentado. A realização do exercício te ajudará a reter mais ainda o que está sendo apresentado ao longo da disciplina. Bons estudos...

1. Explique a frase: Acoplamento e Complexidade Quanto Menor Melhor.

2. Por que o empacotamento de dados e funções na Orientação a Objetos é vantajoso?

3. Faça um comparativo entre a Programação Funcional e a Programação Orientada a Objetos.

4. Complete o conteúdo da cada frase abaixo:

- a. A abstração controla a complexidade...
- b. A encapsulação facilita as mudanças...
- c. A hierarquia (grafo de herança) permite a construção de software...
- d. O software orientado a objetos é mais fácil de manter porque sua estrutura....

5. Para a situação **(a)** temos um programa com um alto acoplamento e um fraco encapsulamento. Explique como seria possível corrigir esse problema estrutural.

Um programa com as Classes **Carro**, **Vendedor** e **Consumidor**. Na implementação atual, cada classe usa diretamente os atributos de **Carro** consumo e quantidade de litros para efetuar o cálculo da *Km* (Regra1:  $Km = consumo * quantidade\ de\ litros$ ). Suponha que seja necessário fazer a manutenção da Regra1:  $Km = consumo * quantidade\ de\ litros * 0.01$ .



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the central portion of the page below the header and above the footer.

**ASSOCIAÇÕES  
E  
COMPOSIÇÃO**

**CAPÍTULO 5**



## Anotações

Naturalmente, objetos em uma aplicação estão relacionados. Uma forma de associação é a hierarquia de objetos que é representada através da herança entre Classes. Assim, se uma classe `Ponto3D` herda de uma classe `Ponto`, pode-se afirmar que um objeto de `Ponto3D` está abaixo na hierarquia em relação a um objeto do tipo `Ponto`.

Outra forma de associação são as referências ou relacionamentos. Os relacionamentos determinam a forma de associação entre os objetos. O relacionamento entre uma classe `Empregado` e uma classe `Departamento` indicará como os objetos dessas duas classes se relacionarão. Por exemplo, pode-se enunciar que um `Empregado` trabalha em somente um `Departamento` e que em um `Departamento` podem trabalhar vários `Empregados` (ilustrar). Existem alguns tipos de associações que são estruturais, também conhecidas de composição. Quando um objeto é responsável por outros objetos ou é composto por outros objetos tem-se um tipo de associação denominada de agregação. Por exemplo, um `Circulo` é composto por dois `Pontos` que representam o seu centro. Assim, quando se enuncia que um objeto é “parte-de”, tem-se a ocorrência de composição ou agregação. Existem dois tipos de agregação: simples ou composta. A agregação simples mantém a semântica de composição “parte-de”, mas o objeto pode ser compartilhado, ou seja, fazer “parte-de” mais de um objeto. Na agregação composta, somente um objeto possui os objetos agregados. Uma forma fácil de fazer a diferença entre agregação simples e composta é que, na agregação composta, os objetos agregados estão no mesmo ciclo de vida do objeto que os agrega. Por exemplo, um objeto `Pedido` é composto por `Itens`, caso o `Pedido` encerre o seu ciclo de vida, os `Itens` também serão encerrados. A composição de objetos é um tipo de relação muito utilizada em projetos dessa natureza. Como no mundo real, onde os objetos são compostos por outros objetos, na orientação a objetos isto não é diferente. Na prática, observa-se que em projetos OO o uso de composição ocorre quase quem 100% dos projetos. Este livro traz na seção X diversos exemplos práticos de composição devido a sua importância.

## 5.1 COMPONENTE

O conceito de componente de software é entendido como uma composição de classes que juntas formam uma unidade para cumprir certas responsabilidades. Componentes devem possuir uma interface pública para a comunicação com os blocos de código externos (não confundir interface homem máquinas, ou seja, telas, menus com interface de um componente que é a forma padronizada de comunicação com o mesmo). Eles têm como principal característica o funcionamento como caixas pretas, em que o desenvolvedor não precisa, necessariamente, conhecer os detalhes de sua implementação. Esta diretiva permite a redução da complexidade na produção de um software visto que o desenvolvedor se abstrai da complexidade do algoritmo interno do componente. Outro ponto relaciona-se com a qualidade do código encapsulado no componente, pois, como o bloco de código é usado em diversos projetos, por princípio, este código já está homologado e bem testado. Assim, através da componentização e dos conceitos de Programação Orientada a Objetos, é possível a construção de projetos com alto grau de reutilização e compartilhamento de código, gerando uma redução no ciclo de desenvolvimento e facilitando as futuras manutenções corretivas e evolutivas.

## 5.2 APLICANDO CONCEITOS RELACIONADOS COM COMPOSIÇÃO

Para a melhor absorção da prática associada à composição, veremos uma introdução sobre coleções em Java. Ressalta-se que o conteúdo que será apresentado nesta seção não esgota o assunto. Assim, para um maior grau de aprofundamento no tema, recomendam-se leituras complementares.

Antes de detalhar a temática de coleções, é importante entender por que este assunto está tão associado como composição. Para compormos objetos com outros, é necessário saber como constituir as relações entre eles. Quando um objeto é composto por somente um objeto de uma outra classe, o mapeamento é feito através de uma associação ou referência. De forma aplicada, é

necessário que a classe que é composta por outra possua um atributo do seu tipo. Quando temos uma relação de uma classe com  $n$  objetos de outra classe, a forma de mapeamento é através de um atributo multivalorado. Por exemplo, se uma Agenda está associada com  $n$  objetos contatos da classe Contato, a forma de mapeamento é a criação na classe Agenda de um atributo multivalorado com uma coleção de contatos, ou seja, este atributo terá  $n$  referências para objetos do tipo contato. A figura 28 apresenta graficamente os dois tipos de composição entre objetos.

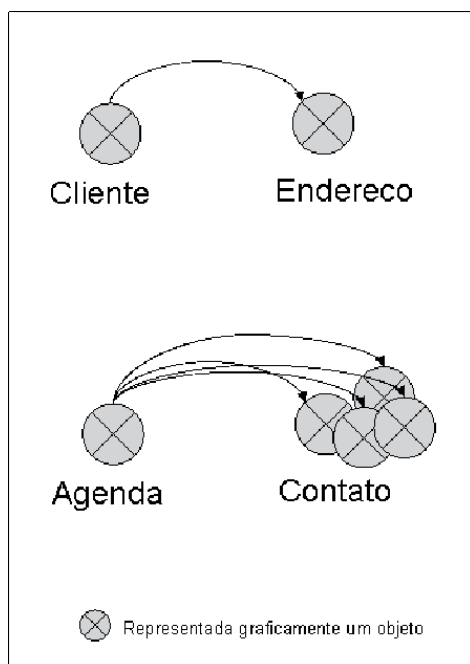


Figura 28 - Representação gráfica da relação de composição entre objetos  
Fonte: Os autores

Para a implementação de atributos multivalorados, pode-se utilizar vetores como, por exemplo, ilustra a figura 29. Neste caso, o nosso atributo multivalorado é a variável  $c$  que pode referenciar até três objetos do tipo contato.

### 5.3 COLEÇÕES (COLLECTION)

Apesar deste tipo de implementação nos possibilitar a criação de atributos multivalorados, o esforço para desenvolver todos os algoritmos de inserção, remoção, alteração, recuperação e ordenação de objetos é muito grande. Assim, vamos usar coleções que é um conjunto

```
public class Contato {
    private String nome;
    private String telefone;

    /**
     * @return Returns the nome.
     */
    public String getNome() {
        return nome;
    }
    /**
     * @param nome The nome to set.
     */
    public void setNome(String nome) {
        this.nome = nome;
    }
    /**
     * @return Returns the telefone.
     */
    public String getTelefone() {
        return telefone;
    }
    /**
     * @param telefone The telefone to set.
     */
    public void setTelefone(String telefone) {
        this.telefone = telefone;
    }
}
```

Figura 29 - Classe Contato  
Fonte: Os autores

```
public static void main(String args[]){
    Contato[] c = new Contato[3];
    c[0] = new Contato();
    c[0].setNome("Eduardo");
    c[0].setTelefone("3351-9878");

    c[1] = new Contato();
    c[1].setNome("Pedro");
    c[1].setTelefone("3345-6789");

    c[2] = new Contato();
    c[2].setNome("Maria");
    c[2].setTelefone("8789-7688");

    for (int i=0; i<c.length; i++){
        System.out.println(c[i].getNome()+"/"+c[i].getTelefone());
    }
}
```

Figura 30 - Exemplificação de um vetor para a Classe Contato  
Fonte: Os autores

de bibliotecas de classe e interfaces disponíveis no Java que nos permite reutilizar algoritmos para a composição de grupos de objetos. De forma sucinta, coleções (também chamadas de "container") é um objeto que agrupa múltiplos objetos. As coleções são usadas para armazenar, recuperar e manipular dados na memória primária. Para isso, existem vários tipos de coleções que devem ser usadas de acordo com o problema a ser resolvido (por exemplo ArrayList, HashSet, HashMap, etc)

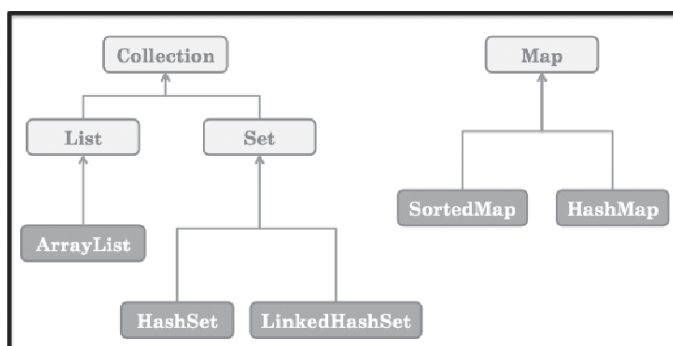


Figura 31- Diagrama de hierarquia de interface de Coleções em Java

Fonte: Os autores

A classe *Collection* é a superclasse mais genérica para as estruturas de armazenamento no Java. Sua interface deve ser implementada por todas as suas classes filhas e seus métodos mais utilizados são os seguintes:

- **boolean** *add(Object)* – O método *add* pode adicionar qualquer objeto em sua estrutura, retornando um tipo *boolean* para indicar se o mesmo foi adicionado com sucesso..
- **boolean** *addAll(Collection)* – O método *addAll* pode adicionar qualquer coleção de objetos em sua estrutura, retornando um tipo *boolean* para indicar se a mesma foi adicionada com sucesso..
- **boolean** *remove(Object)* – O método *remove* pode remover qualquer objeto que esteja armazenado em sua estrutura, retornando um tipo *boolean* para indicar se o procedimento foi realizado com sucesso.
- **boolean** *removeAll(Collection)* – O método *removeAll* pode remover uma coleção de objetos que esteja armazenado em sua estrutura, retornando um tipo *boolean* para indicar se a mesma foi removida com sucesso..
- **boolean** *contains(Object)* – O método *contains* verifica se o objeto passado está armazenado em sua coleção e retorna um tipo *boolean* para indicar a resposta..
- **boolean** *containsAll(Collection)* – O método *containsAll* verifica se a coleção de objetos passada está armazenada em sua coleção e retorna um tipo *boolean* para indicar a resposta..
- **boolean** *isEmpty()* – O método *isEmpty* indica, através de um retorno do tipo *boolean*, se a coleção está vazia..

- **void** *clear()* – O método *clear* limpa a coleção, removendo todos os objetos armazenados..

Dentre as filhas da interface *Collection*, podemos citar *List* e *Set*, pois são duas das coleções mais utilizadas em Java. A interface *List* visa a criação de estruturas de armazenamento que implementem a ideia de uma lista encadeada, como, por exemplo, a classe concreta *ArrayList*. As estruturas que herdam a classe *List* devem implementar, entre outros e além dos já citados da classe *Collection*, os seguintes métodos:

- **boolean** *add(int, Object)* – Esta sobrecarga do método *add* pode adicionar qualquer objeto em sua estrutura, a partir de um índice (número inteiro) que servirá para inserir o objeto na posição desejada, tendo 0 como sua primeira posição. Também haverá um retorno do tipo *boolean* para indicar se o mesmo foi adicionado com sucesso.
- **Object** *remove(int)* – Esta sobrecarga do método *remove* pode remover a partir de uma posição, informada no índice que é recebido por parâmetro. Neste caso, o retorno será o objeto (tipo *Object*) removido.
- **Object** *get(int)* – O método *get* pode localizar um objeto, também, de acordo a sua posição. Neste caso, o retorno será o objeto (tipo *Object*) desejado.
- **int** *indexOf(Object)* – O método *indexOf* serve para identificar o índice de um objeto, ou posição na qual o mesmo foi armazenado. O retorno deste método é o índice do objeto informado no parâmetro.

Já a interface *Set* visa a criação de estruturas que não permitam a duplicidade de seus elementos, como, por exemplo, a classe concreta *HashSet*. As estruturas que herdam a classe *Set* devem implementar, entre outros, os métodos já citados da classe pai *Collection*.

Utilizando uma coleção, podemos implementar o conceito de composição de elementos em orientação a objetos. Quando identificamos, por exemplo, que uma turma é composta por vários alunos, a implementação correta da classe *Turma* deve prever uma variável do tipo *Set* que armazene vários tipos da classe *Aluno*, já que o mesmo aluno não pode estar presente na sala duas vezes. Utilizando uma variável do tipo *Set* para armazenar a coleção de alunos, estamos garantindo a unicidade da relação. A seguir um exemplo de implementação desse possível problema:

```

public class Turma {
    private String disciplina;
    private Set<Aluno> alunos;

    public String getDisciplina() {
        return disciplina;
    }

    public void setDisciplina(String disciplina) {
        this.disciplina = disciplina;
    }

    public Set<Aluno> getAlunos() {
        return alunos;
    }

    public void setAlunos(Set<Aluno> alunos) {
        this.alunos = alunos;
    }
}

public static void main(String[] args) {
    Turma turmaA = new Turma();
    turmaA.setAlunos(new HashSet<Aluno>());

    Aluno jose = new Aluno();
    turmaA.getAlunos().add(jose);

    Aluno pedro = new Aluno();
    turmaA.getAlunos().add(pedro);
}

```

Figura 32 - Exemplo de implementação Turma  
Fonte: Os autores

No caso de um problema que envolva a modelagem de uma linha de produção, onde a montagem de um produto envolve a organização em fila dos itens que compõem o produto, deve-se utilizar a classe List, pois um mesmo item pode ser utilizado mais de uma vez durante o processo de montagem do mesmo produto, tendo a necessidade de estar presente duas ou mais vezes na mesma lista. A seguir um exemplo de implementação desse problema utilizando a classe List:

```

public class Produto {
    private String partNumber;
    private List<Item> itens;

    public String getPartNumber() {
        return partNumber;
    }

    public void setPartNumber(String partNumber) {
        this.partNumber = partNumber;
    }

    public List<Item> getItens() {
        return itens;
    }

    public void setItens(List<Item> itens) {
        this.itens = itens;
    }
}

public static void main(String[] args) {
    Produto produtoA = new Produto();
    produtoA.setItens(new ArrayList<Item>());

    Item porca = new Item();
    produtoA.getItens().add(0, porca);

    Item parafuso = new Item();
    produtoA.getItens().add(1, parafuso);
}

```

Figura 33 - Exemplo de implementação Produto  
Fonte: Os autores

As coleções podem ser percorridas facilmente no Java através da estrutura de repetição For. A seguir, temos um exemplo de implementação que trata disso:

```

public static void main(String[] args) {
    Produto produtoA = new Produto();
    produtoA.setItens(new ArrayList<Item>());

    Item porca = new Item();
    produtoA.getItens().add(0, porca);

    Item parafuso = new Item();
    produtoA.getItens().add(1, parafuso);

    for (Item it : produtoA.getItens()) {
        int alt = it.getAltura();
        int larg = it.getLargura();
    }
}

```

Figura 34 - Exemplo de implementação For  
Fonte: Os autores

Além de List e Set, outra estrutura de armazenamento utilizada no Java pode ser a da classe Map. Esta trabalha de forma diferente de uma coleção, porém utiliza uma dentro de sua estrutura. Ao invés de adicionar o objeto solto dentro da classe um Map, sempre irá vincular esse objeto com uma chave, que deverá ser outro objeto.

A seguir temos os principais métodos de uma classe Map:

- `Object put(Object, Object)` – Este método é responsável por adicionar dois elementos na estrutura de armazenamento; o 1º é a chave que servirá de índice para o 2º, que é o valor que se deseja armazenar.

- `Object remove(Object)` – O método remove pode remover a partir da chave, que é recebida por parâmetro. Neste caso, o retorno será o objeto (tipo *Object*) removido.

- `Object get(Object)` – O método get pode localizar um objeto de acordo com a chave passada. Neste caso, o retorno será o objeto (tipo *Object*) desejado.

- **boolean** `containsKey(Object)` – Este método permite ao programador verificar se uma determinada chave (passada no parâmetro) já está sendo utilizada para indexar algum valor. Para isso, é utilizado um retorno do tipo *boolean*.

- **boolean** `containsValue(Object)` - O método *containsValue* é utilizado para verificar se um objeto já foi adicionado ao Map e retornar um boolean para indicar a resposta.

- `Set<Object> keySet()` – O método *keySet* retorna a coleção que armazena todas as chaves que estão sendo utilizadas.

- `Collection<Object> values()` – Este método retorna em uma coleção todos os objetos que estão sendo armazenados no *Map*.

A seguir, temos um exemplo de utilização da estrutura *Map*:

```
public class Turma {
    private String disciplina;
    private Map<String, Aluno> alunos;

    public String getDisciplina() {
        return disciplina;
    }

    public void setDisciplina(String disciplina) {
        this.disciplina = disciplina;
    }

    public Map<String, Aluno> getAlunos() {
        return alunos;
    }

    public void setAlunos(Map<String, Aluno> alunos) {
        this.alunos = alunos;
    }
}

public static void main(String[] args) {
    Turma turmaA = new Turma();
    turmaA.setAlunos(new HashMap<String, Aluno>());

    Aluno jose = new Aluno();
    jose.setMatricula("ABC123");
    turmaA.getAlunos().add(jose.getMatricula(), jose);

    Aluno pedro = new Aluno();
    pedro.setMatricula("EDF456");
    turmaA.getAlunos().put(pedro.getMatricula(), pedro);
}
```

Figura 35 - Exemplo de estrutura MAP

Fonte: Os autores

As estruturas de armazenamentos também podem ser utilizadas para outros fins que não sejam a composição de classes. Tudo vai depender do problema em questão e de como a solução deve ser implementada.

Nesta seção, aborda-se o tema reutilização. O objetivo é discutir os vários níveis ☐ análise, projeto ou código ☐ de reutilização. Em seguida, faz-se o estudo de dois tipos de reutilização: Framework Orientado a Objeto e Padrões de Projeto.

## 5.4 MOTIVAÇÃO PARA A REUTILIZAÇÃO DE SOFTWARE

O desenvolvimento de projetos de software não é uma tarefa fácil. Os projetos, geralmente, têm que ser robustos, atuar sobre problemas complexos e estar implantados em curto prazo (“time-to-market”).

A reutilização é reconhecida como um importante modo para se alcançar um aumento na produtividade em projetos de software, pois possibilita agregar funcionalidades pré-existentes na produção de novos softwares. Ressalte-, também, a possibilidade de programadores iniciantes criarem softwares complexos através da utilização de padrões de projetos descritos por programadores mais experientes.

Além do mais, a reutilização de pedaços de software garante uma maior qualidade ao projeto, visto que os blocos a serem reutilizados já estão testados e validados por uma ou mais aplicações. Portanto, o direcionamento no desenvolvimento de um projeto de software é para que não se construa nada que já exista e que possa ser reutilizado.

Entretanto, é necessário produzir softwares genéricos e extensíveis que possam ser aplicados a uma gama de aplicações. Ao longo dos anos, muitas metodologias surgiram na engenharia de software com o intuito de reduzir a complexidade e aumentar a produtividade no desenvolvimento. A Orientação a Objetos é reconhecida hoje como o principal paradigma para atender a essa redução da complexidade.

## 5.5 REUTILIZAÇÃO E ORIENTAÇÃO A OBJETOS

Reutilização de software é o grande desafio dos projetistas de software. A engenharia de software vem evoluindo substancialmente para ir ao encontro dessa necessidade. Dentre as diversas metodologias existentes, Orientação a Objetos (OO) é o principal paradigma para a construção de software reutilizável. Apesar do grande sucesso do paradigma OO, nem sempre os resultados, em relação à reutilização, são obtidos de forma fácil.

As vantagens da Orientação a Objetos, como paradigma para a reutilização de software, são bem conhecidas.

As abstrações podem corresponder às coisas do domínio do problema

O nível é mais natural.

É mais fácil comunicar-se com o usuário ou “domain expert” na linguagem dele.

Os mesmos objetos existem em todas as fases e uma notação única (objetos) facilita, portanto, a integração entre fases de desenvolvimento (passar de uma fase para a outra).

É mais fácil entender o domínio do problema quando este é quebrado em pedaços: gerenciamento da complexidade através da modularização.

O mesmo pode ser dito no domínio do computador (projetando e programando com objetos).

A abstração controla a complexidade (escondendo algo através da separação da interface e da implementação).

A encapsulação facilita as mudanças (através do isolamento).

A hierarquia (grafo de herança) permite uma melhor reutilização.

Um dos mecanismos da Orientação a Objetos para alcançar a reutilização de código é o uso de biblioteca de classes que encapsulam determinadas funcionalidades que possam ser reutilizadas. Embora classes sejam unidades de código reutilizáveis, uma classe para cumprir sua funcionalidade, geralmente, tem dependências com outras classes. Para a realização de tarefas complexas é necessário que um grupo de classes trabalhem conjuntamente, caracterizando, assim, um componente de software.

O conceito de componente de software pode ser entendido como uma composição de classes que juntas formam uma unidade para cumprir certas responsabilidades. Componentes devem possuir uma interface pública para a comunicação com os blocos de código externos. Eles têm como principal característica o funcionamento como ‘caixas pretas’, onde o desenvolvedor não precisa, necessariamente, conhecer os detalhes de sua implementação.

Através do uso de componentes, a reutilização popularizou-se. Um exemplo clássico são as ferramentas RAD (“Rapid Application Development”) gráficas (Visual Basic, Delphi, Centura, etc), que fornecem componentes visuais aplicáveis nas construções de interfaces gráficas. Os componentes visuais (VBX, OCX, etc) disponíveis nessas ferramentas proporcionam um aumento na produtividade de desenvolvimento de sistemas comerciais.

Além da reutilização de código, existem outros tipos de reutilização. Por exemplo, a reutilização de projeto atua não somente no patamar de código, como também nos projetos conceitual e lógico.

A questão que se coloca é:

Como ‘encapsular’, em bibliotecas ou em componentes, comportamentos genéricos para um domínio de aplicação<sup>1</sup>, que possam ser usados em software de mesmo domínio<sup>2</sup>, só que com características específicas?

A próxima seção trata de Framework, uma técnica de desenvolvimento de software que permite a construção de projetos reutilizáveis em todos os níveis.

## 5.6 FRAMEWORK ORIENTADO A OBJETOS

A reutilização em um projeto de sistemas deve contemplar todos os níveis de um projeto e não se restringir somente à codificação, ou seja, deve englobar análise, projeto, codificação e testes.

Framework provê reutilização em alto nível. Portanto, através da técnica de projeto de software framework, é possível construir projetos extensíveis, em que a reutilização está focada desde o domínio da aplicação.

É importante destacar que framework não é alternativa à Orientação a Objetos ou a componentes, mas sim, trata-se de um recurso adicional de projeto para reutilizar algo mais do que código.

Na verdade, é difícil imaginar a construção de um framework que não seja OO. A Orientação a Objetos possui características como abstração, ‘encapsulamento’, herança e colaboração de classes que viabilizam a sua criação. Da mesma forma, um framework é construído com base em componentes.

Na sequência, tratamos o tema Framework com mais detalhes, envolvendo o conceito, as diferenças entre framework e bibliotecas de classes, o projeto de um framework, e a construção de aplicações baseadas em frameworks. Por fim, discutimos a importância do tema.

## 5.7 CONCEITO DE FRAMEWORK

Na literatura existente, são várias as definições de framework. Nesta seção, vamos enfatizar as duas que, na nossa opinião, representam melhor o conceito de framework.

Iniciamos com uma definição genérica:

“Framework na sua essência pode ser definido como um conjunto de blocos de software que os programadores podem usar, estender, ou ‘customizar’, com pouco esforço, para um domínio específico de problema.” [IBM 99].

Já uma definição mais técnica determina que

“Um framework é um conjunto de classes colaborativas abstratas e concretas, que pode ser usado como um ‘template’ (gabarito) para resolver uma família de problemas relacionados. Ele é usualmente estendido através de subclasses, para obter-se o comportamento específico de uma aplicação.” [LAR00]

É importante enfatizar que existe uma diferença substancial entre framework e biblioteca de classes.

## 5.8 FRAMEWORK VERSUS BIBLIOTECA DE CLASSES

A compreensão do conceito de framework relaciona-se com o entendimento de sua diferença com biblioteca de classes. A confusão entre os dois conceitos é uma incompreensão clássica entre desenvolvedores de software.

Framework ainda pode ser visto como uma espécie de biblioteca, só que o controle do fluxo de chamadas é bi-direcional, ou seja, tanto uma aplicação pode chamar métodos do framework como o framework pode invocar métodos da aplicação. Isto é diferente de uma biblioteca de classes stricto sensu, em que o fluxo de chamadas é uni-direcional, da aplicação para a biblioteca. A possibilidade de o framework chamar

<sup>1</sup> Contexto em que uma aplicação está inserida

<sup>2</sup> Uma fronteira formal que define um particular assunto ou área de interesse [LAR00].

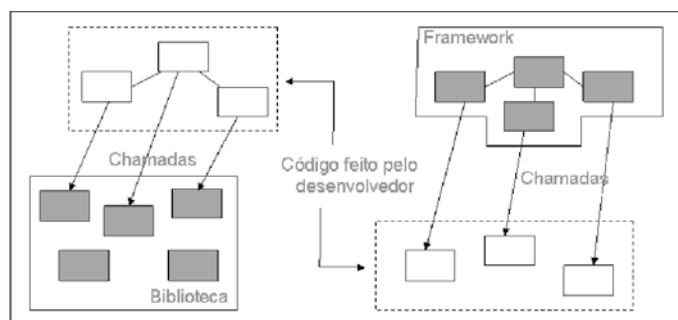


Figura 36: Diferença no fluxo de controle entre Framework e bibliotecas de classes

Fonte: [LAN95]

métodos da aplicação se dá através de chamadas “dynamic binding”, implementadas nas subclasses da extensão do framework para a aplicação.

A figura 32 ilustra duas aplicações. A primeira aplicação possui classes que fazem chamadas às classes pertencentes a uma biblioteca de classes. A segunda aplicação foi criada a partir de um Framework. Neste caso, são as classes do Framework que fazem as chamadas às classes da aplicação. A construção de um framework não é uma tarefa trivial. Passemos a discuti-la.

## 5.9 FRAMEWORK E REUTILIZAÇÃO DE SOFTWARE

Ao projetar um Framework, é imprescindível levar em conta que sua estrutura deve servir para a construção de diversos sistemas, que pertencem a um mesmo domínio de aplicação. Este projeto genérico deve capturar e “encapsular” todas as funcionalidades comuns às futuras aplicações que serão desenvolvidas, como ilustra a figura 33.

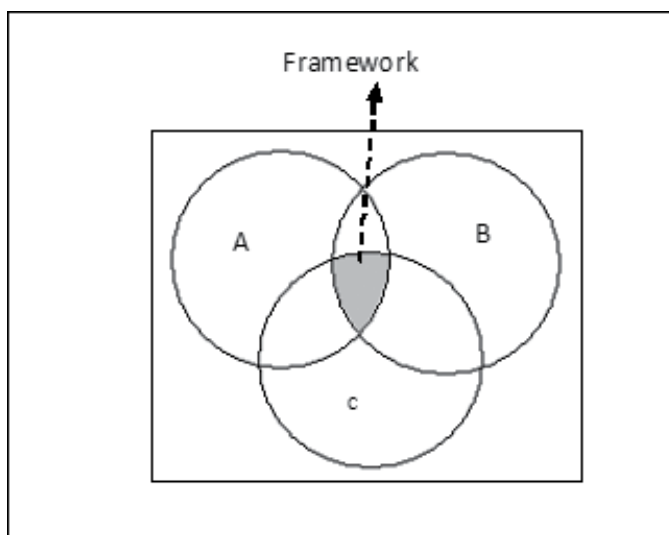


Figura 37 - Interseção de três aplicações

Fonte: Os autores

A figura 33 sugere três aplicações A, B e C, representadas por círculos. Estas aplicações possuem funcionalidades específicas e funcionalidade comuns. As funcionalidades comuns □ framework □ estão representadas no gráfico pela área hachurada que é a interseção dos três círculos. Quanto maior a interseção entre as aplicações, mais funcionalidades poderão estar “encapsuladas” no framework.

Um framework deve ser projetado para ser usado através do princípio “Hollywood Principle”, isto é, “Não nos chame, nós iremos chamar você”. As classes desenvolvidas nas aplicações irão receber mensagens das classes do framework (figura 33). Assim, é possível que as classes definidas nas aplicações façam o resto do trabalho específico que o framework deixou em aberto.

Ainda ao projetar-se um framework, deve-se construir uma aplicação quase que completamente, faltando apenas “pedaços” para serem completados com as funcionalidades específicas das futuras aplicações. Mais precisamente, a arquitetura do framework deve ser tal que possibilitará às aplicações que vierem a ser construídas utilizar a sua infraestrutura. A figura 34 dá uma ideia disto. A aplicação é a “soma” do framework e do “bloco” com as extensões que “encapsulam” as funcionalidades específicas.

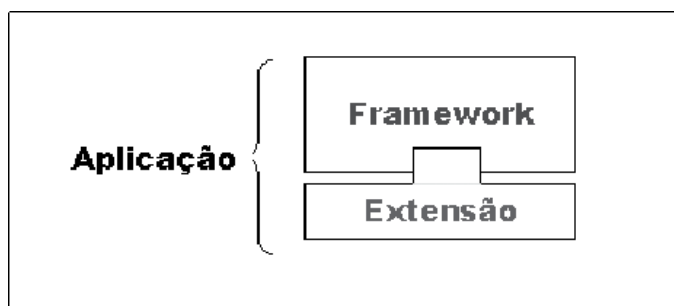


Figura 38 - Aplicação construída utilizando a infraestrutura de um Framework

Fonte: Os autores

Pode-se observar também na figura 34 que o 'bloco' do framework é 'maior' que o 'bloco' da extensão, sugerindo que foi alcançado um alto grau de reutilização, o que é desejável.

Por fim, um framework deve ser tratado como um produto que necessita de documentação e suporte. Como produto, ele deve ser planejando para ser distribuído e sofrer manutenções constantes.

## 5.10 CONSTRUINDO APLICAÇÕES A PARTIR DE UM FRAMEWORK

Para construir aplicações a partir de um framework o desenvolvedor, preliminarmente, deve:

Verificar se o framework atende às necessidades da aplicação.

Analisar o grau de complexidade para incorporá-lo à aplicação que será gerada.

Distinguir qual é a camada (subsistema) que é fixa, ou seja, o framework, e qual é a camada variável (o subsistema que será desenvolvido).

Fica óbvio então que, para começar, o framework a ser reutilizado deve ser muito bem projetado, e dispor de uma excelente documentação. A documentação deve descrever em detalhes os passos a ser seguidos para a construção de uma aplicação através do framework, incluindo:

- que classes devem ser estendidas;
- que métodos devem ser implementados;
- que novas classes devem ser criadas.

Na ótica do usuário 'implementador' (aquele que vai estender o framework), o desenvolvimento da aplicação, na prática, implica numa 'instanciação' do framework. Por conseguinte, o desenvolvedor terá de estender por herança algumas classes abstratas ou utilizar composição em outras. Como foi dito antes, ele deve utilizar-se da documentação que detalha o que é necessário para estender o Framework. É como utilizar 'ganchos' existentes no framework, que permitem 'plugar' funcionalidades. Tratam-se de blocos de código que serão adicionados ao framework, para que o sistema fique habilitado a executar funcionalidades específicas da aplicação.

Ressalta-se que, ao desenvolver uma aplicação a partir de um framework, está-se, na verdade, reutilizando o projeto do framework como um todo, incluindo os níveis conceitual, lógico e físico.

## 5.11 FRAMEWORK "A BOLA DA VEZ"

O conceito de Framework não é algo recente e desconhecido na engenharia de software. Desde do final da década de 80, muitas pesquisas nesta área já tinham sido realizadas [WEI88] [TAL93].

Então, porque só mais recentemente o conceito de framework tem sido largamente aceito? Isto se deve:

À maturidade das linguagens orientadas a objeto (Java, C++, etc);

À demanda por projetos de software mais complexos e reutilizáveis;

Aos novos recursos da engenharia de software como padrões de projeto [GAM94], processo iterativo incremental [JAC98], UML [LAR00], etc.

Dentre os recursos citados, padrões de projeto têm uma importância significativa e podem ser de grande valia para os desenvolvedores de frameworks. A próxima seção trata de padrões de projeto, com o foco em framework.



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

**PADRÕES  
DE PROJETO**

**CAPÍTULO 6**



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the central portion of the page below the header and above the footer.

O bom desenvolvimento de um software orientado a objeto depende de alguns fatores:

Uma boa decomposição do sistema em classes e objetos.

O conhecimento pelos desenvolvedores dos diversos aspectos ligados à orientação a Objeto:

Encapsulação

Reutilização

Flexibilidade

Portabilidade

Desempenho

Um meio para contemplar todos esses requisitos é a utilização de padrões de projeto. Alguns motivos para isto são:

Desenvolvedores iniciantes podem utilizar padrões de projetos catalogados por “experts” para obter conhecimento e experiência na construção de aplicações orientadas a objeto.

Com a boa utilização de padrões de projeto, a comunicação entre desenvolvedores e a manutenção de sistemas torna-se menos complexa.

Padrões de projeto podem ajudar a encontrar abstrações que tornem o software mais flexível e reutilizável.

Cada padrão descreve um problema que ocorre várias vezes em um ambiente de desenvolvimento de software, podendo ser utilizado milhões de vezes em situações diferentes, constituindo-se, assim, num dos cerne da solução [GAM94].

Um padrão de projeto orientado a objeto descreve uma situação em que várias classes cooperam para realizar uma determinada tarefa, criando uma organização específica, ou seja, microarquitecturas de classe e objetos com seus papéis e colaborações. Essa microarquitectura é uma solução genérica a um problema que pode ocorrer em diversas aplicações. Dizemos que um padrão de projeto resolve um problema recorrente.

A definição de um padrão de projeto é ela própria padronizada da seguinte forma (Quadro 2)

| Documentação de Padrões de Projeto |                                                                                                                                                                                                                                                                |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nome                               | Identificador usado para descrever o problema, suas soluções e consequências, em uma ou duas palavras;                                                                                                                                                         |
| Descrição                          | Apresenta o problema e seu contexto;<br>Indica quando devemos aplicar o padrão;<br>Pode apresentar problemas específicos de projeto, ou descrever estruturas de classes ou objetos que geralmente tornam o projeto inflexível (estruturas a evitar, portanto). |
| Solução                            | Descreve as relações, as responsabilidades, as colaborações e os elementos que compõem a solução do projeto;<br>Serve como um gabarito que pode ser utilizado em várias situações.                                                                             |
| Consequências                      | Apresentam os resultados e “trade-offs” da utilização do padrão;<br>Ajudam a avaliar se os resultados obtidos justificam a adoção do padrão;<br>Apresentam os impactos em termos de flexibilidade, extensibilidade e transportabilidade.                       |

Quadro 2 - Documentação de Padrões de Projeto  
Fonte: GAMMA; HELM; JOHNSON; VLISSIDES, 1994

Padrões de Projeto têm tomado dimensões que sobrepõem a parte de projeto físico. Já existem padrões de análise, testes etc. Na próxima seção, tratamos do relacionamento entre padrões de projeto e framework e exemplificaremos o padrão “Template Method”.

## 6.1 PADRÕES DE PROJETO E FRAMEWORK

Num projeto de um framework, padrões de projeto ajudam a encontrar abstrações que tornam o software mais flexível e reutilizável.

Por exemplo, num framework temos o problema de como instanciar objetos de classes que foram “plugadas” no framework. Neste caso, poder-se-ia utilizar o padrão

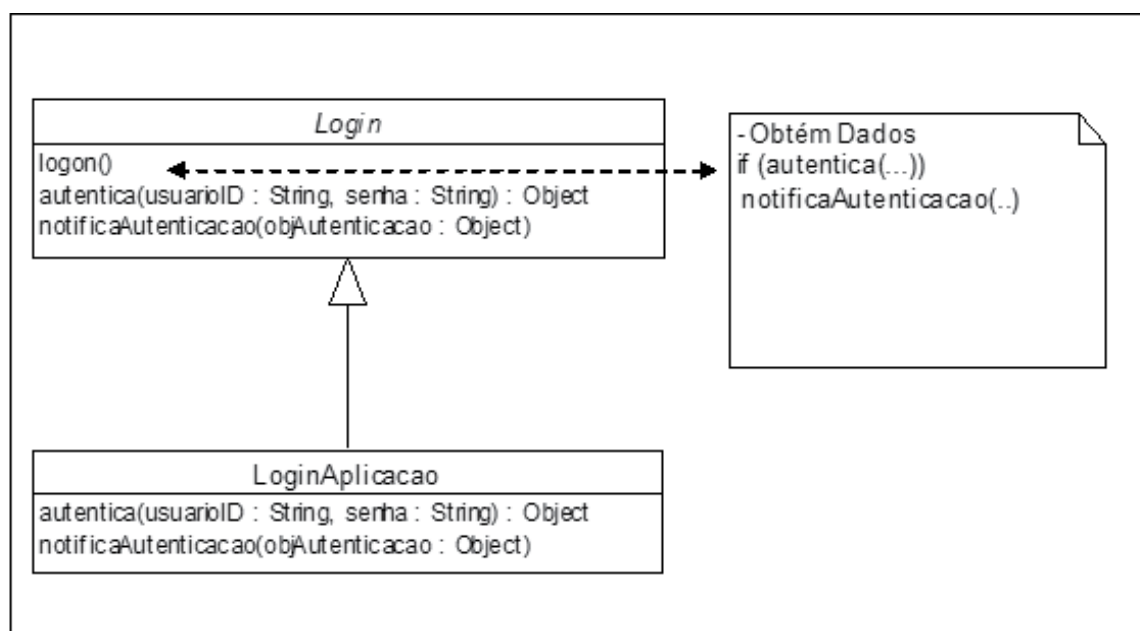


Figura 39 - Classe e subclasse de Login  
Fonte: Os autores

de projeto “Abstract Factory”, que provê uma solução genérica para este tipo de problema. Os padrões de projeto mais comuns utilizados em projetos de framework são: “Façade”, “Command”, “Template Method”, e “Abstract Factory”.

Dentre os padrões citados, “Template Method” é a base para a construção de um framework.

## 6.2 TEMPLATE METHOD

“Template Method” define o esqueleto de um algoritmo para uma operação, transferindo alguns passos desta operação para as subclasses. Este padrão permite que uma subclasse redefina certos passos de um algoritmo, sem alterar a estrutura do mesmo.

Seja um problema prático “Criar uma solução genérica de “Login” que possa ser reutilizada em qualquer projeto.”

A figura 35 ilustra um digrama de classes com a solução do problema enunciado. Define-se uma classe abstrata Login □ estrutura genérica preparada para ser ‘customizada’ pelas futuras aplicações. Esta classe contém um método “logon” □ “Template Method” □ que ‘encapsula’ a lógica do processo genérico de login. Este método fará chamadas aos métodos abstratos

“autentica” e “notificaAutenticacao”. Os métodos abstratos serão implementados pela subclasse concreta LoginAplicacao que estende a classe Login. A classe LoginAliação é responsável pela lógica específica da aplicação que está usando o ‘template’ de Login.

Adota-se uma solução padrão para problemas do tipo exemplificado. A figura 36 apresenta uma microarquitetura genérica, ou seja, servindo para qualquer aplicação que necessite do “Template Method”. Esta microarquitetura é base para construção de um Framework.

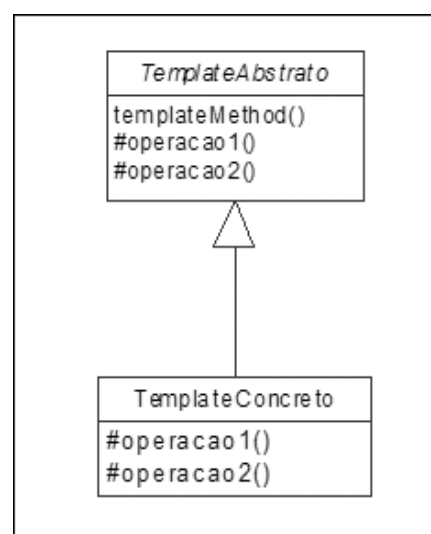


Figura 40 - Padrão Template Method  
Fonte: Os autores

A classe `TemplateAbstrato` é a classe abstrata que contém o comportamento genérico. Na aplicação de Login, equivale a classe `Login`. Esta classe possui um método “`templateMethod`” que encapsula a lógica da solução e faz chamadas a operações abstratas (`#operacao1`, `#operacao2`). Por fim, temos a subclasse `TemplateConcreto` que estende a classe `TemplateAbstrato` para definir as operações, `operacao1` e `operacao2`, com funcionalidades específicas.

A próxima seção trata de um processo de desenvolvimento voltado para a construção de um framework.



#### VOCÊ SABIA?

O Hibernate é um framework para o mapeamento objeto-relacional escrito na linguagem Java, mas também é disponível em .Net com o nome NHibernate. Ele facilita o mapeamento dos atributos entre uma base tradicional de dados relacionais e o modelo objeto de uma aplicação, mediante o uso de arquivos (XML) ou anotações Java. Outros exemplos de frameworks são: Rich Faces, MyFaces, Struts, Spring, JDBC.



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

**O PROCESSO  
UNIFICADO**

# **CAPÍTULO 7**



## Anotações

A tendência é de desenvolvimento de sistemas de software cada vez maiores e mais complexos. Isto se deve à evolução dos computadores, que estão se tornando cada vez mais poderosos, fazendo com que os usuários alimentem mais expectativas em relação a eles. Esta tendência também tem sido influenciada pela expansão do uso da internet e da multimídia.

Esse contexto leva à necessidade de utilização de processos de desenvolvimento que sejam adequados ao dinamismo e às expectativas do mercado. Em nível de implementação, já existe um consenso de que OO é a chave para o desenvolvimento rápido de aplicações complexas. Em relação a análise e projeto, faltava uma notação unificada realmente eficaz. Esta lacuna foi preenchida com o surgimento da linguagem UML □ “Unified Modeling Language” [JAC98]. UML é uma linguagem de modelagem, mas ela não define um processo de desenvolvimento. Como complemento à UML, surgiu uma especificação para um processo de desenvolvimento de software, chamada de Processo Unificado [JAC98].

O Processo Unificado contempla todo o ciclo de vida de um projeto, e guia as equipes de desenvolvimento nas atividades práticas de engenharia de software. Ele tem sua base em três conceitos chaves:

Desenvolvimento orientado por caso de uso.

Ênfase em uma arquitetura, desde o início.

Abordagem iterativa e incremental.

## 7.1 DESENVOLVIMENTO ORIENTADO POR CASO DE USO

A construção de um software tem por objetivo atender às requisições dos usuários. As requisições ditas funcionais no Processo Unificado desembocam nos casos de uso □ pedaços de funcionalidades de sistemas que retornam valores em que os usuários estão interessados. O conjunto de casos de uso forma o diagrama de casos de uso, que descreve o contexto do sistema, com todas as suas entidades externas (Atores), funcionalidades (Casos de Uso) e os seus relacionamentos.

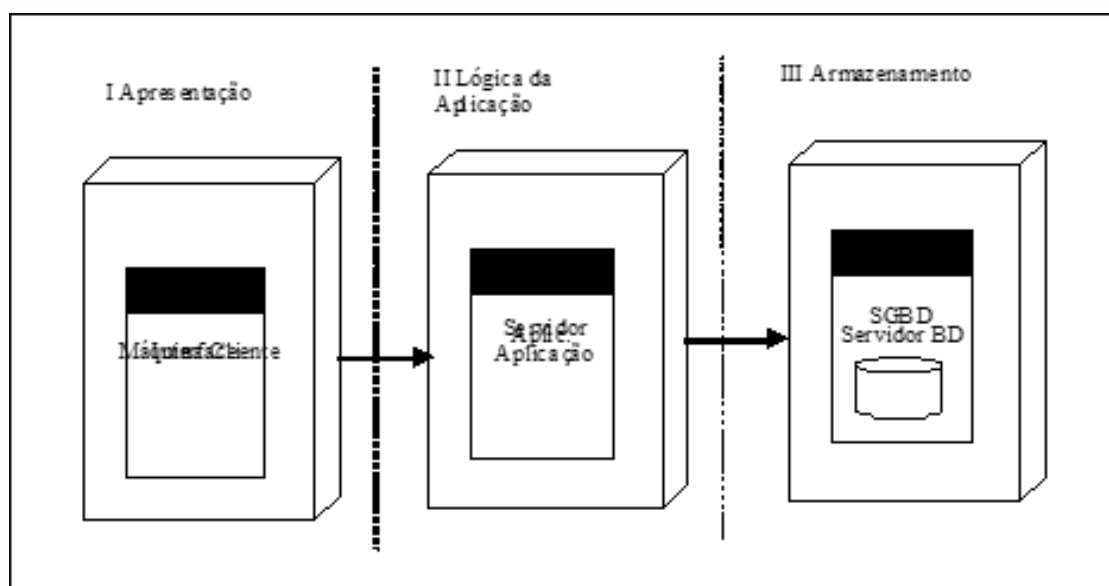


Figura 41 - Visão clássica de um arquitetura em três camadas

Fonte: Os autores

Na fase de análise, os casos de uso captam e validam as funcionalidades com os usuários. As fases seguintes, projeto, implementação e testes, serão norteadas pelos casos de uso.

## 7.2 ÊNFASE NA ARQUITETURA (DESDE A FASE DE ELABORAÇÃO DO PROJETO)

A arquitetura de um software é o mais importante aspecto do sistema, e refere-se à estrutura de subsistemas e de componentes. É um mapa do sistema, com as diferentes partes, suas interações e mecanismos de interconexões. Devido à importância da arquitetura no desenvolvimento de software, o Processo Unificado propõe que ela seja definida o mais cedo possível, quase que em paralelo com a coleta dos requisitos.

Ao projetar-se a arquitetura, vários fatores são importantes:

A plataforma que o software vai rodar □ a arquitetura do computador, o sistema operacional, o gerenciador de Banco de Dados, etc.

Considerações de desenvolvimento □ linguagem de programação, internacionalização, etc

Integração com sistemas legados e requisitos não-funcionais □ performance, confiabilidade, tipo de interface etc

A figura 37 apresenta um arquitetura comum para sistemas de informação. A arquitetura está dividida em três camadas: apresentação, lógica da aplicação e armazenamento. Na figura em questão, cada camada está distribuída em máquinas diferentes (representadas pelas caixas). Listamos alguns elementos das camadas.

I Apresentação – menus, botões, janelas etc.

II Lógica da Aplicação – objetos e regras de negócio.

III Persistência – mecanismos de armazenamento persistente (XML, tabelas em um SGBD, etc).

## 7.3 ABORDAGEM ITERATIVA E INCREMENTAL

Os processos de desenvolvimento convencionais têm por premissa ciclos de desenvolvimento em ‘cascata’, ou seja, faz-se necessário o término de uma fase para o início de outra. Neste processo, o software só é “lançado” quando as fases de análise, projeto, implementação e testes (do software como um todo) já foram concluídas. Para sistemas que têm um ciclo de desenvolvimento extenso, temos o problema da primeira versão ser disponibilizada muito tempo após a coleta dos requisitos, que podem ter sofrido mudanças ao longo desse período.

O Processo Unificado iterativo e incremental difere dos processos convencionais, pois baseia-se num ciclo de desenvolvimento curto, em que as versões (incrementos) são disponibilizadas ao longo do ciclo de desenvolvimento do software.

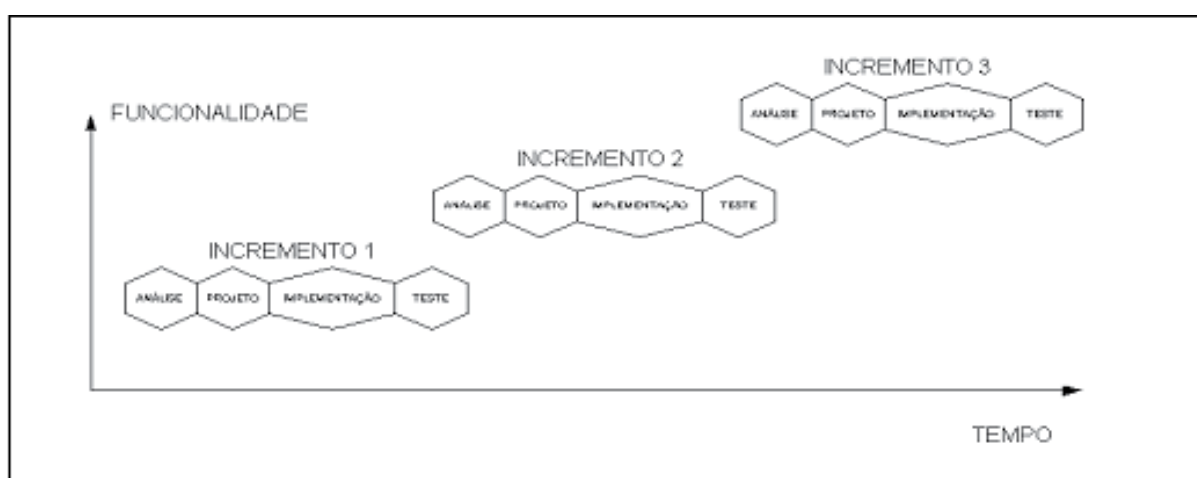


Figura 42 - Processo Iterativo e Incremental  
Fonte: Os autores

Para a construção das versões, são feitas várias iterações. As iterações são uma sequência de etapas e análise, projeto, implementação, testes e implantação que detalham e ampliam o modelo. Ao final de um ciclo de interações, temos uma versão do sistema.

A vantagem desse processo é que, ao longo de todo o processo, podem ser feitas avaliações a respeito do cumprimento dos requisitos funcionais, desempenho, confiabilidade, cronograma, recursos utilizados, entre outros. Esta forma de trabalhar diminui riscos que são inerentes ao processo de desenvolvimento de um software. A figura 38 exemplifica graficamente o Processo Unificado iterativo e incremental [JAC98]

O gráfico da figura 38 apresenta a evolução dos incrementos no tempo. Cada incremento para ser construído passa por todo o ciclo de iterações. Ao final de um ciclo de iterações, temos um incremento com mais funcionalidades.

Durante o processo de desenvolvimento dos incrementos, deve-se utilizar técnicas de reutilização de software, para aumentar a produtividade dos desenvolvedores. A obtenção de bons resultados é dependente da experiência de profissionais que desenvolvem projetos (com o pensamento voltado para a reutilização), e do emprego de técnicas de reutilização, como Framework e Padrões de Projeto.



## Anotações

A large, empty rounded rectangle with a light gray border and a subtle drop shadow, intended for taking notes. It occupies the majority of the page below the header.

## REFERENCIAS

[GAM94] GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1994.

[IBM 99] INTERNATIONAL BUSINESS MACHINES (IBM). **Building Object-Oriented Frameworks**. White Paper, 1999.

[JAC98] JACOBSON, I.; BOOCH, G.; RUMBAUGH, J. **The Unified Software Development Process**. Addison-Wesley, 1998.

[LAN95] LANDIN, N.; NIKLASSON, A. **Development of Object-Oriented Framework**. Conden:Lutedx, 1995.

[LAR00] LARMAN, C. **Utilizando UML e padrões: Uma Introdução à Análise e ao Projeto Orientados a Objetos**. Bookman, 2000.

[TAL93] TALIGENT INC. **Leveraging Object-Oriented Frameworks**. A Taligent White Paper, 1993.

[WEI88] WEINAND, A; GAMMA, E.; MARTY, R. ET + + - An Object+Oriented Application Framework in: C + + . In: **Proceedings** of the OOPSLA'88, 1988 .



## Anotações